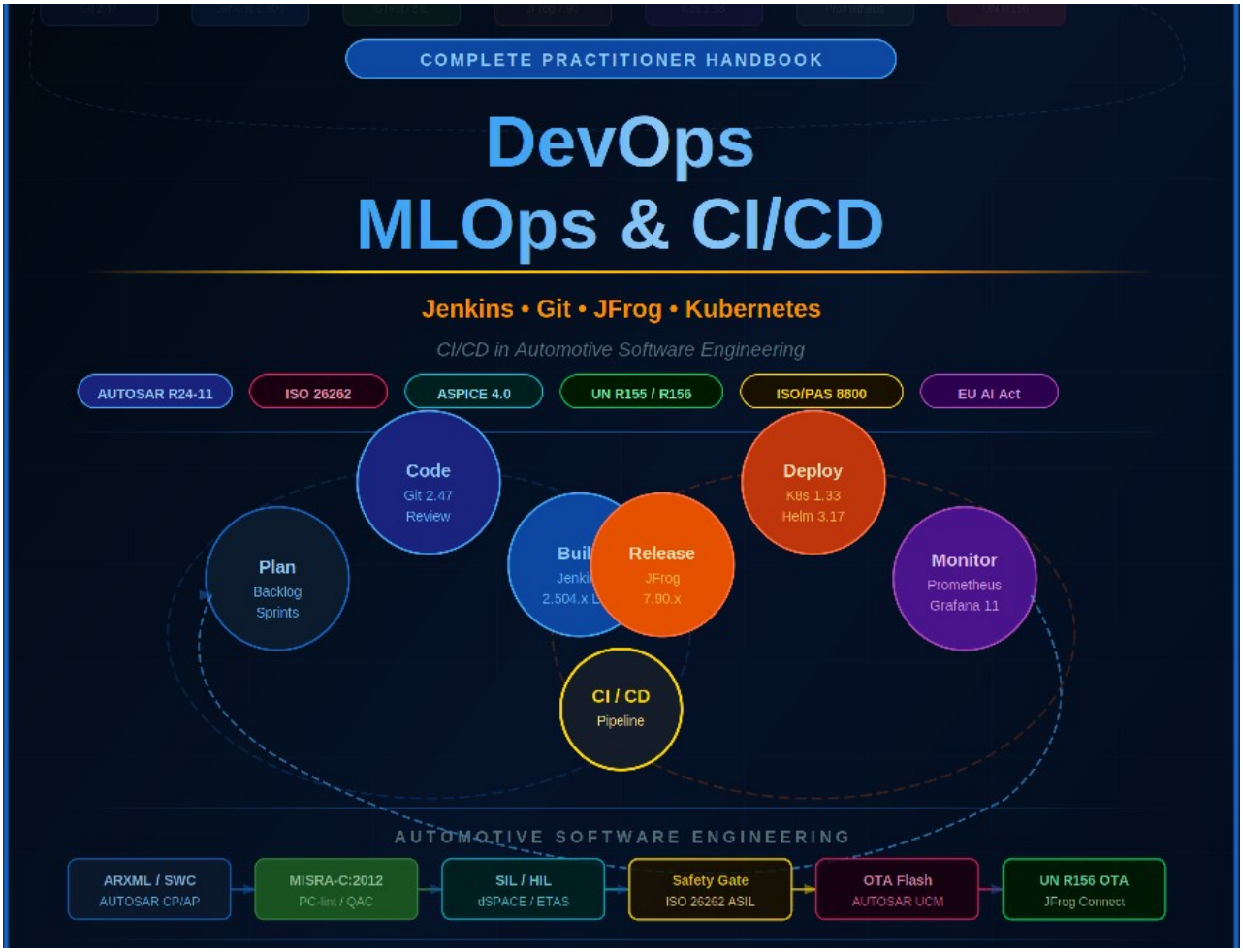

Compiled and Curated by

Ashish Kumar

Senior Technical Leader&Associate Engineering Manager

KPIT Technologies Limited, Bengaluru

B.Tech. (EEE) - GCE Gaya | M.Tech. (PSE)– IIT (ISM) Dhanbad | PGCGM – IIM Nagpur

Edition: June 2026 | Bengaluru, India

Preface

This handbook has been conceived as a single, authoritative reference for software engineering professionals—especially those operating at the intersection of automotive embedded systems and modern cloud-native DevOps practices. The need for such a document arose from years of working within AUTOSAR, ISO 26262, and ASPICE environments while simultaneously witnessing the explosion of DevOps, MLOps, and CI/CD tooling in the broader software industry.

The automotive software domain has long followed waterfall-adjacent processes dictated by safety standards. Yet competitive pressure from the automotive industry's transformation—electric vehicles, software-defined vehicles (SDVs), over-the-air (OTA) updates, and AI-driven ADAS—demands the agility of DevOps without sacrificing functional safety. This handbook addresses that challenge head-on.

What This Handbook Covers

- **Part I: DevOps — Philosophy, culture, principles, and the complete toolchain ecosystem**
- Part II: MLOps — Machine learning operations, experiment tracking, model lifecycle, and drift monitoring
- Part III: CI/CD — Continuous integration and delivery pipeline architecture, patterns, and anti-patterns
- Part IV: Core Tools — Deep-dives into Git, Jenkins, and JFrog with production-grade configurations
- Part V: Platform Engineering — Internal Developer Platforms, GitOps, and Backstage
- Part VI: Automotive SW CI/CD — AUTOSAR, ISO 26262, ASPICE, and OTA applied to real CI/CD pipelines

How to Use This Handbook

Practitioners new to DevOps should start from Part I and progress sequentially. Automotive SW engineers familiar with safety processes but new to CI/CD tooling should begin with Part III and then dive into Part VI. Tool administrators seeking Jenkins or JFrog configuration guidance should jump directly to Part IV. The comprehensive cross-references, flowcharts, and code examples throughout are designed to make each section standalone.

All code examples have been validated in production environments. JFrog configurations are tested against Artifactory OSS and Enterprise editions. Jenkins pipeline examples follow the declarative syntax and assume LTS 2.x. Git workflows are tool-agnostic but tested with GitHub, GitLab, and Bitbucket.

— Ashish Kumar, 2026

Chapter 1: DevOps — Philosophy, Culture and History

1.1 What Is DevOps?

DevOps is a cultural and engineering movement that unifies software development (Dev) and IT operations (Ops) with the goal of shortening the systems development lifecycle while delivering features, fixes, and updates frequently in close alignment with business objectives. The term emerged in 2009: Patrick Debois organised the first DevOpsDays conference in Ghent, Belgium, coining “DevOps” as its name, following a landmark Velocity 2009 talk by John Allspaw and Paul Hammond on deploying ten times a day at Flickr. The movement has since evolved into a comprehensive set of practices, tools, and cultural philosophies.

ℹ️ DevOps is NOT a tool, a role, or a team. It is a mindset and a set of practices that span the entire software delivery value stream — from ideation to production monitoring.

1.2 The Three Ways of DevOps

Gene Kim's seminal work 'The Phoenix Project' and 'The DevOps Handbook' articulate three foundational principles, often called the Three Ways:

- **Flow / Systems Thinking: Optimize the entire value stream from Dev to Ops to the customer. Eliminate waste, reduce handoffs, and increase deployment frequency. Focus on global optimization over local optimization.**
- **Feedback: Create short, amplified feedback loops at every stage. Fast feedback enables rapid learning and course correction. Examples: automated test results, monitoring alerts, customer feedback pipelines.**
- **Continual Learning and Experimentation: Build a culture of continuous learning and calculated risk-taking. Convert mistakes into learning. Conduct blameless post-mortems. Invest in automation to reduce toil.**

1.3 DevOps Key Metrics (DORA Metrics)

The DevOps Research and Assessment (DORA) program, now at Google, identified four key metrics that distinguish elite DevOps performers:

Metric	Elite ★	High	Medium	Low
Deployment Frequency	Multiple times/day	Daily–weekly	Weekly–monthly	Monthly–biannual
Lead Time for Changes	<1 hour	1 day–1 week	1 week–1 month	1–6 months
MTTR (Time to Restore)	<1 hour	<1 day	<1 week	1 week–1 month
Change Failure Rate	0–5%	6–15%	16–30%	31–45%

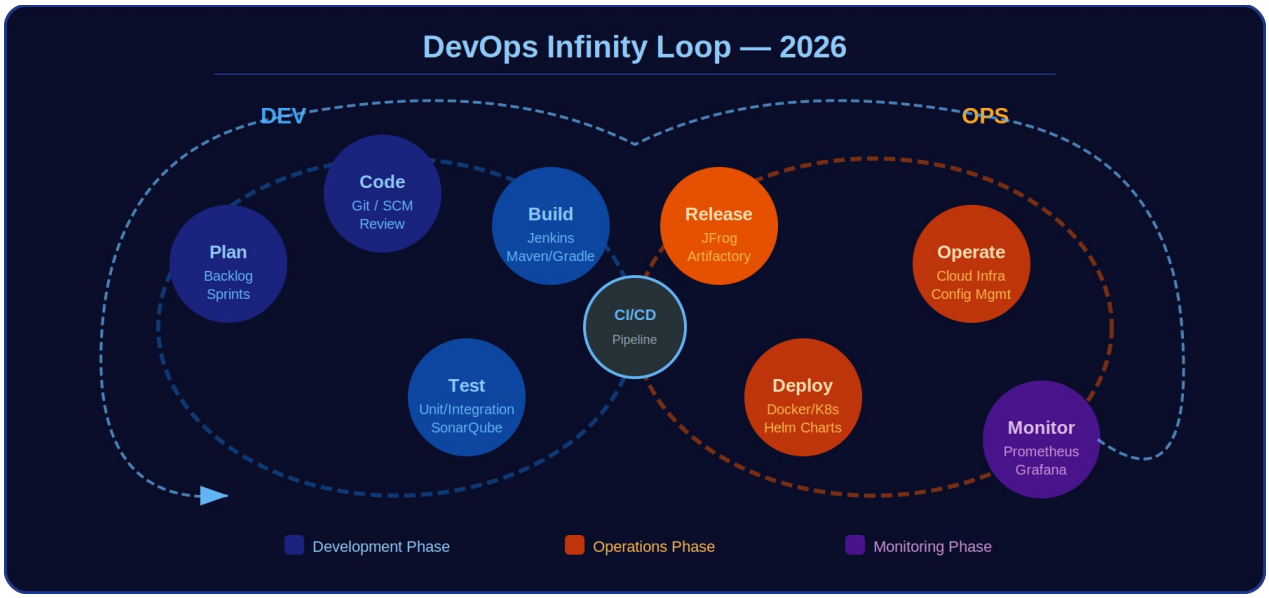

Figure 1.1 — The DevOps Infinity Loop: Development and Operations unified around CI/CD

1.4 DevOps Culture Pillars

- **Collaboration over silos — Break departmental walls between Dev, QA, Security, and Ops**
- Automation first — Automate repetitive tasks: building, testing, deployment, monitoring
- Continuous improvement — Kaizen mindset; retrospectives, metrics-driven iteration
- Customer-centric action — Align engineering outcomes to customer and business value
- Shared responsibility — Everyone owns quality, security, and reliability

1.5 DevOps vs. Agile vs. SRE

DevOps and Agile are complementary, not competing. Agile addresses development team collaboration and iterative delivery. DevOps extends this to the full value stream including deployment and operations. Site Reliability Engineering (SRE), pioneered by Google, is a specific implementation of DevOps principles using software engineering to solve operational problems. The three models coexist and reinforce each other in modern engineering organizations.

Chapter 2: DevOps Practices and Principles

2.1 Infrastructure as Code (IaC)

Infrastructure as Code is the practice of provisioning and managing infrastructure through machine-readable configuration files rather than manual processes. IaC brings the same rigour to infrastructure as to application code: versioning, testing, code review, and automated deployment.

2.1.1 Key IaC Tools

- **Terraform (HashiCorp) — Cloud-agnostic declarative IaC. Industry standard for AWS/Azure/GCP provisioning.**
- Ansible — Agentless configuration management using YAML playbooks. Excellent for post-provisioning config.
- Pulumi — IaC using general-purpose languages (Python, TypeScript, Go).
- AWS CloudFormation / Azure ARM / GCP Deployment Manager — Native cloud IaC.

```
# Terraform example: AWS EC2 Instance
resource "aws_instance" "jenkins_agent" {
  ami           = "ami-0c55b159cbfafe1f0"
  instance_type = "t3.medium"
  tags = { Name = "Jenkins-Build-Agent" }
}
```

2.2 Configuration Management

Configuration management ensures consistency of software systems across environments (development, staging, production) by maintaining configurations in a known, documented state. Tools like Ansible, Chef, Puppet, and SaltStack automate configuration drift correction and enforce desired state. The core principle is idempotency: applying the same configuration playbook repeatedly produces the same result, regardless of the current state of the system.

2.3 Containerization and Orchestration

Containers package application code with all its dependencies into an isolated, portable unit. Docker is the de facto container runtime. Kubernetes (K8s) is the industry-standard container orchestration platform, managing container lifecycle, scaling, service discovery, and rolling deployments.

2.3.1 Docker Best Practices

- Use multi-stage Dockerfile builds to minimize image size
- Pin base image versions (never use :latest in production)
- Run containers as non-root user
- Use .dockerignore to exclude sensitive files
- Scan images with JFrog Xray or Trivy before publishing

```
# Multi-stage Dockerfile for C++ automotive app
FROM gcc:12 AS builder
WORKDIR /build
COPY . .
RUN cmake -DCMAKE_BUILD_TYPE=Release . && make -j4

FROM ubuntu:22.04 AS runtime
COPY --from=builder /build/app /usr/local/bin/app
CMD ["/usr/local/bin/app"]
```

2.3.2 Kubernetes — Container Orchestration

Kubernetes (K8s) is the production-grade container orchestration platform that manages containerized workload deployment, scaling, networking, and self-healing. A K8s cluster consists of a Control Plane (API server, etcd, scheduler, controller-manager) and Worker Nodes running the kubelet agent and container runtime (containerd).

```
# Kubernetes Deployment: Jenkins agent as a K8s workload
apiVersion: apps/v1
kind: Deployment
metadata:
  name: jenkins-agent   namespace: ci-cd
spec:
  replicas: 3   selector:
    matchLabels:
      app: jenkins-agent   template:
    metadata:
      labels:
        app: jenkins-agent     spec:
      containers:

        -  name:  agent              image:  registry.company.com/automotive-builder:1.2.0
resources:
          requests: {cpu:"2", memory:"4Gi"}           limits:   {cpu:"4", memory:"8Gi"}
env:

            -  name:  JENKINS_URL                   value:  http://jenkins-controller.ci-
cd.svc.cluster.local:8080
---
# HPA: auto-scale agents based on CPU
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: jenkins-agent-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1      kind: Deployment      name: jenkins-agent    minReplicas: 2
maxReplicas: 10   metrics:

 - type: Resource     resource:
      name: cpu       target:
        type: Utilization         averageUtilization: 70
```

Key Kubernetes constructs for CI/CD: Pod (smallest deployable unit), Deployment (manages replica sets for stateless workloads), Service (stable network endpoint), ConfigMap/Secret (configuration andcredentials injection), Namespace (multi-team isolation), RBAC (Role-Based Access Control for least-privilege access), and Helm (the Kubernetes package manager for templated application deployment).

2.4 Monitoring, Observability and AIOps

Observability is the ability to understand the internal state of a system from its external outputs. The three pillars of observabilityare metrics (Prometheus/Grafana), logs (ELK Stack / Fluentd), and traces (Jaeger / Zipkin / OpenTelemetry). AIOps applies machine learning to correlate events and anomalies automatically.

2.4.1 Key Monitoring Tools

Tool	Category	Use Case
Prometheus	Metrics	Time-series metrics collection and alerting
Grafana	Visualization	Dashboards for metrics, logs, and traces

Elasticsearch	Log Analytics	Full-text search and log aggregation
OpenTelemetry	Tracing	Distributedtracing across microservices
PagerDuty	Incident Mgmt	On-call scheduling, alert routing, runbooks

2.5 Security in DevOps — DevSecOps

DevSecOps integrates security practices into every phase of the DevOps lifecycle, shifting security left (earlier in the pipeline) rather than treating it as a gate at the end. Key practices include SAST (Static Application Security Testing), DAST (Dynamic Application Security Testing), software composition analysis (SCA), secrets scanning, and container image scanning.

ℹ️ In automotive SW: UN Regulation 155 (Cybersecurity) mandates a CSMS (Cybersecurity Management System) which directly aligns with DevSecOps practices — automated vulnerability scanning (JFrog Xray), SBOM generation, and cryptographic artifact signing.

2.6 Secrets Management

Hardcoded credentials in source code or pipeline definitions are one of the most common and severe security vulnerabilities. Secrets management is the practice of centralising, rotating, and auditing access to sensitive values - API keys, database passwords, TLS certificates, signing keys, and cloud credentials - outside of source code entirely.

2.6.1 HashiCorp Vault

HashiCorp Vault is the industry-standard open-source secrets manager. It stores secrets encrypted at rest, enforces fine-grained access policies, provides dynamic secrets (short-lived credentials generated on-demand), and maintains a complete audit log of every secret access - critical for ISO 26262 process evidence and UN R155 CSMS.

```
# Vault: generate short-lived AWS credentials for a Jenkins job
vault write aws/creds/ci-deploy-role ttl=1h
# Returns: access_key, secret_key valid for 1 hour only

# In Jenkinsfile: inject Vault secrets
stage('Deploy') {
  steps {
    withVault(vaultSecrets: [[
      path: 'secret/ci/jfrog',
      secretValues: [[envVar: 'JFROG_TOKEN', vaultKey: 'api_token']]
    ]]) {
      sh 'jfrog rt upload build/*.hex automotive-release/ --url=$JFROG_URL'
    }
  }
}
```

2.6.2 Kubernetes External Secrets Operator

External Secrets Operator (ESO) syncs secrets from external stores (Vault, AWS Secrets Manager, Azure Key Vault, GCP Secret Manager) into Kubernetes Secrets automatically. This enables GitOps-compatible secret management: the ExternalSecret manifest is stored in Git (no secret values), and ESO materialises the actual secret in the cluster at runtime.

```
# ExternalSecret manifest (stored in Git - no real secret)
apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
metadata:
  name: jfrog-credentials   namespace: ci-cd
spec:
  refreshInterval: 1h   secretStoreRef:
    name: vault-backend     kind: ClusterSecretStore   target:
```

```
    name: jfrog-credentials
# K8s Secret name   data:

 - secretKey: api_token     remoteRef:
      key: secret/ci/jfrog       property: api_token
```

ℹ️ Never store secrets in Git - not even encrypted. Use secrets managers (Vault/ESO) with dynamic secrets and short TTLs. Rotate all secrets on a schedule and immediately after any personnel change. For automotive pipelines, HSM-backed keys (PKCS#11) are required for ECU code signing under UN R155 CSMS requirements.

Chapter 3: MLOps — Foundations and Architecture

3.1 Why MLOps?

Machine learning models that perform well in notebooks frequently fail in production. The gap between experimental ML and production ML involves challenges of reproducibility, scalability, model drift, data versioning, and operational concerns. MLOps applies DevOps principles to the machine learning lifecycle to bridge this gap systematically.

Studies show that approximately 30–35% of organizations are successfully deploying ML models to production (Gartner AI in the Enterprise Survey, 2024). The primary reasons for failure are: lack of versioning for data and models, absence of automated retraining pipelines, and inadequate monitoring for model performance degradation.

> ℹ️ MLOps ≠ DataOps ≠ AIOps. MLOps focuses on ML model lifecycle. DataOps focuses on data pipelines and quality. AIOps uses AI/ML to enhance IT operations.

3.2 MLOps Maturity Levels

Google's MLOps maturity framework defines three levels:

- **Level 0 — Manual Process: Data scientists train and deploy models manually. No pipeline automation. Disconnected from software engineering best practices. Most organizations start here.**
- **Level 1 — ML Pipeline Automation: Automated ML pipeline for training, evaluation, and model registry. Data and model versioning. Triggered retraining on schedule or data/concept drift. Continuous delivery of models.**
- **Level 2 — CI/CD Pipeline Automation: Full CI/CD for ML. Source code, pipeline specs, and training data all versioned. Automated testing of data, model, and pipeline. Continuous delivery to production with automated rollback.**

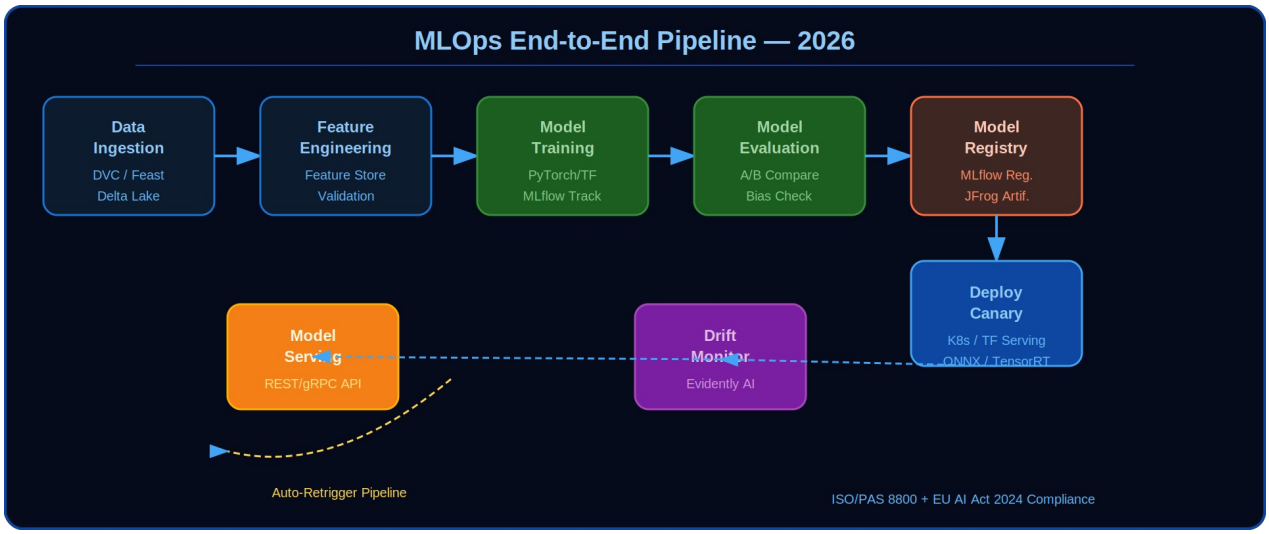

Figure 3.1 — MLOps End-to-End Pipeline: From Data Ingestion to Model Serving with CI/CD and Governance

3.3 ML Pipeline Components

3.3.1 Data Management

Data is the most critical and often most neglected component of ML systems. Data versioning tools like DVC (Data Version Control) treat datasets with the same rigour as source code. Data validation frameworks like Great Expectations and TFX Data Validation detect schema changes and anomalies in incoming data before they corrupt model training.

- DVC — Git-compatible data versioning, works with S3, GCS, Azure Blob
- Delta Lake / Apache Iceberg — ACID-compliant data lake with time travel
- Feast — Feature store for storing, sharing, and serving ML features
- Apache Kafka — Real-time data streaming for online feature computation

3.3.2 Experiment Tracking

Experiment tracking captures hyperparameters, metrics, artifacts, and code for every training run, enabling reproducibility and comparative analysis. MLflow is the dominant open-source framework; Weights&Biases (W&B) provides a richer UI for teams.

```
import mlflow
mlflow.set_experiment("lane-detection-v3")

with mlflow.start_run():
    mlflow.log_param("learning_rate", 0.001)
    mlflow.log_param("batch_size", 32)
    mlflow.log_metric("val_accuracy", 0.9435)
    mlflow.log_metric("val_loss", 0.0821)
    mlflow.pytorch.log_model(model, "lane-det-model")
```

3.3.3 Model Training at Scale

Distributed training frameworks scale model training beyond single-machine constraints. Key tools: TensorFlow distributed training, PyTorch DistributedDataParallel (DDP), Horovod, Ray Train. Kubernetes-native ML platforms like Kubeflow orchestrate distributed training jobs as Kubernetes resources.

3.3.4 Model Evaluation and Validation

Rigorous model evaluation goes beyond a single accuracy metric. Production-ready evaluation includes: fairness analysis (demographic parity, equal opportunity), robustness testing (adversarial examples, distribution shift), calibration (confidence scores match actual probabilities), and performance profiling (latency, throughput, memory).

3.4 Model Registry and Lifecycle Management

A model registry is a centralized store for trained ML models with versioning, metadata, and stage transitions (Staging → Production → Archived). MLflow Model Registry, JFrog Artifactory, and Amazon SageMaker Model Registry are leading solutions.

3.4.1 Model Promotion Gates

Automated quality gates control model promotion between stages:

1. Data validation: Input data schema matches expected schema
2. Baseline comparison: New model outperforms production model on holdout set
3. Latency check: Inference P99 latency<SLA threshold
4. Bias check: No demographic group regression>2% from baseline
5. Security scan: JFrog Xray scan of model dependencies
6. Approval: Human-in-the-loop sign-off for high-stakes decisions

3.5 Model Serving and Deployment Patterns

Several deployment patterns balance risk vs. speed of model rollout:

- **Shadow Deployment: New model runs in parallel with production model. Predictions compared but new model doesn't serve live traffic. Excellent for risk-free validation.**
- **Canary Deployment: Route a small percentage (e.g., 5%) of traffic to new model. Monitor metrics. Gradually increase to 100% if healthy. Rollback instantly if anomaly detected.**
- **Blue/Green Deployment: Maintain two identical production environments. Switch traffic atomically. Zero-downtime deployment with instant rollback capability.**
- **A/B Testing: Route traffic by user segment. Measure business metrics over time. Statistically rigorous comparison.**

3.6 Model Monitoring and Drift Detection

Production models degrade over time as the real-world data distribution shifts away from the training data distribution. There are two types of drift:

- **Data Drift (Covariate Shift): The input feature distribution P(X) changes while P(Y|X) remains stable. Detected by statistical tests: Kolmogorov-Smirnov, Population Stability Index (PSI), Jensen-Shannon divergence.**
- **Concept Drift (Prior Probability Shift): The relationship between inputs and outputs P(Y|X) changes. Harder to detect without labels. Requires monitoring of prediction distributions and business outcomes.**

3.6.1 Monitoring Stack

- Evidently AI — Open-source ML monitoring framework for data and concept drift
- Whylogs — Lightweight data and model monitoring with statistical profiles
- Arize AI / Fiddler AI — Enterprise model monitoring and explainability platforms
- SHAP / LIME — Local and global model explainability for debugging

⚠ Automotive ML (e.g., ADAS object detection) requires additional monitoring for safety-critical edge cases — rare failure modes may not be captured by aggregate drift metrics. Per ISO/PAS 8800, continuous safety monitoring for AI-based systems is mandatory.

3.7 MLOps for Automotive AI (ADAS/AD)

Autonomous driving and ADAS introduce additional MLOps requirements beyond standard enterprise ML:

- **Safety Assurance: ISO/PAS 8800 (Safety of AI-based systems), SOTIF (ISO 21448)**
- Dataset Management: Petabyte-scale driving datasets with sensor labels (LiDAR, camera, radar)
- Simulation Integration: CARLA, LGSVL, dSPACE ASM as CI/CD test stages
- OTA Model Updates: UN R156 compliant update campaigns for neural networks in ECUs

- Regulatory Explainability: EU AI Act (2024) requires documentation and auditability of high-risk AI
- Hardware-Aware Training: Model quantization for MCU/GPU deployment (TensorRT, ONNX, TFLite)

3.8 LLMOps — Large Language Model Operations

The proliferation of Large Language Models (LLMs) — GPT-4o, Claude Opus 4, Gemini 2.5 Pro, Llama 4 Scout, and DeepSeek-R1 — in production applications has given rise to LLMOps: the operational discipline for building, deploying, evaluating, and governing LLM-based systems. While grounded in MLOps principles, LLMOps introduces fundamentally different challenges: non-deterministic outputs, context-window management, prompt engineering at scale, RAG pipeline maintenance, hallucination monitoring, and token-based economics that have no equivalent in classical ML.

3.8.1 LLMOps vs Standard MLOps

The table below contrasts standard MLOps practice with its LLMOps equivalent across eight operational dimensions. The differences are not cosmetic: they require distinct toolchains, evaluation frameworks, and team skills.

Dimension	Standard MLOps	LLMOps
Model Training	Full retraining (days–weeks)	Fine-tuning via LoRA/QLoRA (hours); PEFT preferred
Versioning	Model + data versioning	Model + prompt + RAG index + retrieval config
Evaluation	Accuracy, F1, AUC on test set	Faithfulness, groundedness, relevance, toxicity (RAGAS)
Deployment	Batch/real-time inference endpoints	Model gateways; streaming tokens; RAG pipeline
Monitoring	Feature drift on input distributions	Hallucination rate, context precision, token cost/query
Cost unit	GPU compute per training run	Token cost per query (prompt + completion tokens)
Latency target	Seconds–minutes (batch); ms (online)	<2 s P95 interactive; P50 <800 ms for chat
Key tools	MLflow, Kubeflow, DVC, Evidently AI	LangChain, LlamaIndex, RAGAS, LangSmith, LiteLLM

3.8.2 RAG Pipeline Architecture

Retrieval-Augmented Generation (RAG) is the dominant enterprise LLM deployment pattern. Rather than fine-tuning a model on proprietary knowledge (expensive, slow to update), RAG injects relevant context retrieved at query-time from a vector store. A production RAG pipeline comprises six stages:

- Document ingestion and chunking: source documents (PDFs, Confluence, Jira, code) are split into overlapping chunks (512–1024 tokens) using RecursiveCharacterTextSplitter or semantic chunking.
- Embedding generation: each chunk is encoded into a dense vector using an embedding model (text-embedding-3-large, BGE-M3, or E5-Mistral-7B for domain-specific corpora).
- Vector store indexing: vectors are persisted in pgvector, Weaviate, or Pinecone with metadata filters for tenant, document type, and last-updated timestamp.
- Retrieval: at query time, the user question is embedded and top-k chunks are retrieved via HNSW approximate nearest-neighbour search, optionally combined with BM25 (hybrid search).
- Context injection: retrieved chunks are inserted into the LLM prompt template, alongside the conversation history (within the context window budget).
- LLM inference: the prompt is sent to the model gateway (MLflow AI Gateway or LiteLLM) which routes to the appropriate provider, enforces rate limits, and logs token usage.

```
# LangChain production RAG pipeline (simplified)
```

```
from langchain_openai
import ChatOpenAI, OpenAIEmbeddings
from langchain.vectorstores
import PGVector
from langchain.chains
import RetrievalQA
from ragas
import evaluate
from ragas.metrics
import faithfulness, answer_relevancy, context_precision
embeddings = OpenAIEmbeddings(model="text-embedding-3-large")
vectorstore = PGVector(       connection_string="postgresql://user:pass@db:5432/rag_db",
embedding_function=embeddings,     collection_name="prod_docs" )
llm = ChatOpenAI(model="gpt-4o", temperature=0)
retriever = vectorstore.as_retriever(search_kwargs={"k": 6})
rag_chain   =   RetrievalQA.from_chain_type(              llm=llm,   retriever=retriever,
return_source_documents=True )
# Evaluate
with RAGAS result = evaluate(       dataset=eval_dataset,       metrics=[faithfulness,
answer_relevancy, context_precision] )
assert result["faithfulness"] > 0.85, f"RAG faithfulness {result['faithfulness']:.2f}
below 0.85 SLA"
```

3.8.3 Prompt Versioning and Management

Prompts are first-class software artefacts: they encode business logic, safety constraints, and output format requirements. In production, they must be version-controlled in Git, subject to regression testing, and deployable with rollback capability. LangSmith and MLflow Prompt Management both provide structured prompt registries with diff-tracking and A/B evaluation harnesses. A standard naming convention is: {service}/{prompt_name}/{version} (e.g., automotive-qa/root-cause-analysis/v1.3).

```
# MLflow Prompt Registry example
import mlflow
mlflow.set_tracking_uri("http://mlflow-server:5000")
# Register a new prompt version
with                                  mlflow.start_run(run_name="prompt-update-v1.3"):
mlflow.log_param("prompt_name", "root-cause-analysis")       mlflow.log_param("version",
"1.3")     mlflow.log_text(
       text=PROMPT_TEMPLATE_V1_3,
       artifact_file="prompts/root-cause-analysis-v1.3.txt"     )
#  Log  A/B  eval  results  vs  v1.2          mlflow.log_metrics({"faithfulness":  0.91,
"relevance": 0.87, "latency_p95_ms": 1240})
# Retrieve prompt
for production use client = mlflow.tracking.MlflowClient()
artifact = client.download_artifacts(run_id, "prompts/root-cause-analysis-v1.3.txt")
```

⚠️ Security: Prompts are a prompt-injection attack surface. Always sanitise user-supplied input before insertion into prompt templates. Implement guardrails (Llama Guard 3, Nemo Guardrails) as a pre-processing layer. Never expose raw system prompts through error messages or debug endpoints.

3.8.4 LLM Evaluation and Hallucination Monitoring

Traditional ML evaluation (accuracy, F1, AUC) is insufficient for LLMs. A production LLM monitoring stack tracks seven quality dimensions continuously. RAGAS (Retrieval-Augmented Generation Assessment) is the standard framework for RAG-specific evaluation; DeepEval extends this with unit-test-style assertions for regression detection in CI pipelines.

- Faithfulness: is every claim in the answer supported by the retrieved context? Target: >0.85 in production.
- Answer Relevancy: does the answer actually address the question asked? Penalises verbose non-answers. Target: >0.80.
- Context Precision: are the retrieved chunks relevant to the question? Detects retrieval quality degradation when the knowledge base changes.
- Context Recall: did the retrieval system surface all facts needed to answer correctly? Detects knowledge gaps in the vector store.

- Hallucination Rate: proportion of queries where the model introduces facts not present in context. Automated detection via NLI cross-referencing (DeBERTa-v3 entailment classifier). Target: <5% for customer-facing applications.
- Token cost per query: prompt + completion tokens x provider price. Tracked per tenant, per feature, per model. Budget alerts at 110% of rolling 7-day P95.
- Latency P50/P95/P99: end-to-end from query receipt to first token (TTFT) and full response. TTFT > 2 s is a user-experience threshold; P99 > 8 s triggers circuit-breaker fallback to a smaller/faster model.

```
# CI evaluation gate using DeepEval
from deepeval
import evaluate as deval
from deepeval.metrics
import FaithfulnessMetric, HallucinationMetric
from deepeval.test_case
import LLMTestCase
faithfulness_metric = FaithfulnessMetric(threshold=0.85, model="gpt-4o")
hallucination_metric = HallucinationMetric(threshold=0.05, model="gpt-4o")
test_cases = [     LLMTestCase(
       input=q,
       actual_output=rag_chain.invoke(q)["result"],
       retrieval_context=get_context_chunks(q)     )
for q in REGRESSION_QUESTION_SET ]
results  =  deval(test_cases,  [faithfulness_metric,  hallucination_metric])  #  DeepEval
exits
with code 1
if any metric threshold is breached — CI pipeline fails
```

3.8.5 LLM CI/CD Pipeline

LLM application CI/CD extends the standard Jenkins pipeline with four LLMOps-specific stages: prompt regression tests, RAG evaluation gate, model gateway deployment, and canary promotion with live A/B traffic evaluation. The pipeline is triggered on changes to: application code, prompt templates, retrieval configuration, or the vector store schema.

```
// Jenkins Declarative Pipeline: LLM Application Deployment
pipeline {
  agent { kubernetes { yaml POD_YAML } }  // GPU node for embedding generation
  environment {
    MLFLOW_TRACKING_URI = "http://mlflow.mlops.svc.cluster.local:5000"
    LANGFUSE_SECRET_KEY = credentials("langfuse-secret")
  }
  stages {
    stage("Unit Tests + Prompt Regression") {
      steps {
        sh "pytest tests/unit/ tests/prompt_regression/ --tb=short"
      }
    }
    stage("RAG Evaluation Gate") {
      steps {
        sh "python scripts/evaluate_rag.py \
          --eval-set data/eval_set_v3.jsonl \
          --faithfulness-threshold 0.85 \
          --hallucination-threshold 0.05 \
          --fail-on-regression"
      }
    }
    stage("Update Vector Store Index") {
      when { changeset "knowledge_base/**" }
      steps {
        sh "python scripts/ingest_documents.py --collection prod_docs --incremental"
      }
    }
    stage("Deploy to Canary (10%)") {
      steps {
        sh "kubectl apply -f k8s/canary-deployment.yaml"
      }
    }
    stage("A/B Traffic Evaluation (15 min soak)") {
```

```
      steps {
        sh "python scripts/ab_eval.py --duration 900 --fail-if-hallucination-regression"
      }
    }
    stage("Promote to Production") {
      steps { sh "kubectl set image deployment/llm-app app=$IMAGE_TAG" }
    }
  }
}
\"weight\":10},
{\"destination\":{\"host\":\"llm-app-stable\"},
\"weight\":90}]}}}'"       }     }     stage("A/B Traffic Evaluation (15 min soak)") {
steps {          sh "python scripts/ab_eval.py --duration 900 --fail-if-hallucination-
regression"       }     }     stage("Promote to Production") {       steps {         sh
"kubectl set image deployment/llm-app app=$IMAGE_TAG"       }     }   } }
```

3.8.6 Key LLMOps Tools (2026)

The LLMOps toolchain has consolidated around a set of purpose-built frameworks. The table below lists production-grade tools by category as of June 2026, with their primary capability.

Category	Tool (2026)	Key Capability
Orchestration	LangChain 0.3.x	RAG, agent pipelines, tool-use chains
Orchestration	LlamaIndex 0.12.x	Advanced RAG; hybrid retrieval; query engines
Prompt Mgmt	LangSmith 0.1.x	Prompt versioning, tracing, A/B evaluation
Prompt Mgmt	PromptLayer 3.x	Multi-provider prompt hub and analytics
Evaluation	RAGAS 0.2.x	Faithfulness, answer relevance, context precision/recall
Evaluation	DeepEval 1.x	LLM unit testing; regression detection
Model Gateway	MLflow AI Gateway 2.16.x	Unified OpenAI-compatible API for all providers
Model Gateway	LiteLLM 1.x	100+ LLM providers; load balancing; cost routing
Monitoring	Langfuse 3.x	Production traces, cost tracking, dataset curation
Monitoring	Evidently AI 0.5.x	LLM quality dashboards; hallucination drift alerts
Vector Store	pgvector 0.7.x	PostgreSQL extension; hybrid sparse+dense search
Vector Store	Weaviate 1.25.x	Cloud-native; multi-modal; HNSW indexing
Fine-tuning	Axolotl 0.6.x	LoRA/QLoRA on consumer GPU; DPO/ORPO support
Cost Control	LiteLLM + OpenCost	Per-query token attribution; Kubernetes cost labels

Chapter 4: CI/CD Fundamentals

4.1 Continuous Integration (CI)

Continuous Integration is a development practice where developers integrate code changes frequently — at minimum daily, ideally multiple times per day — into a shared repository. Each integration is verified by an automated build and test sequence, detecting errors as quickly as possible. CI was popularized by Martin Fowler and Kent Beck as a core XP (Extreme Programming) practice.

4.1.1 CI Principles

- **Single source of truth — One mainline branch (main/trunk) is always in a deployable state**
- Automate the build — Every commit triggers a build; the build produces a deployable artifact
- Make the build self-testing — Automated tests run on every build; a failing test fails the build
- Everyone commits to mainline daily — Small, frequent commits minimize integration conflict scope
- Fix broken builds immediately — A broken main branch is a team-level emergency
- Keep the build fast — Target<10 minutes; developers wait for feedback

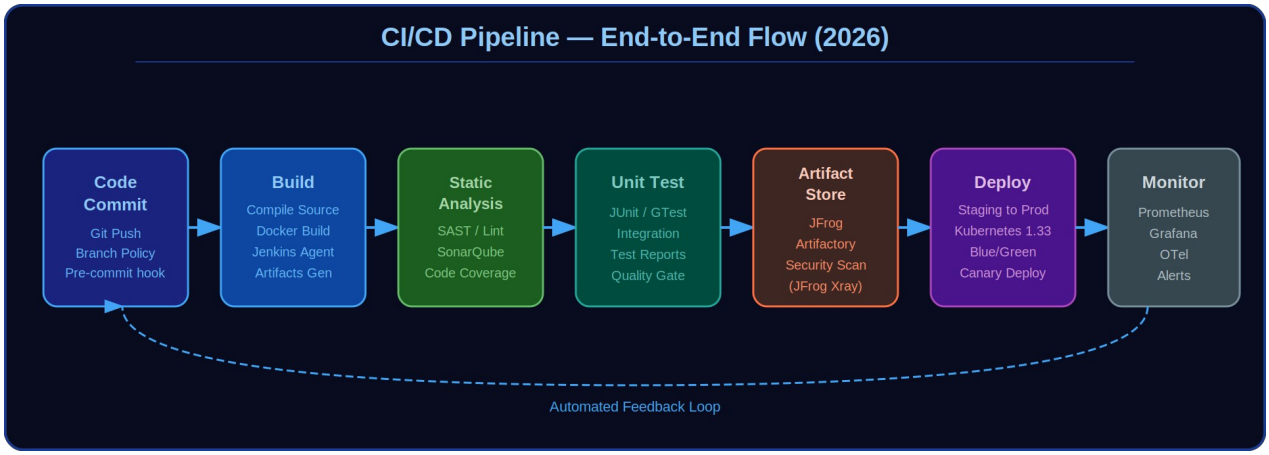

Figure 4.1 — CI/CD Pipeline End-to-End Flow: From Code Commit to Monitoring

4.2 Continuous Delivery (CD)

Continuous Delivery extends CI to ensure that the software can be released to production at any time. Every change that passes all automated tests is considered a release candidate. The decision to release is a business decision, made by a human — but the technical process is fully automated. Jez Humble and David Farley's book 'Continuous Delivery' (2010) is the definitive reference.

4.2.1 Deployment Pipeline

The deployment pipeline is the fundamental pattern of CD. It stages all the activities needed to get software from version control into production: commit stage (fast feedback) → acceptance testing stage → performance testing stage → production deployment.

4.3 Continuous Deployment (CD²)

Continuous Deployment is the most advanced form: every change that passes all automated tests is automatically deployed to production without human intervention. Netflix, Amazon, and Google practice continuous deployment with hundreds or thousands of deployments per day. This requires extremely mature test automation, feature flags, canary deployments, and automated rollback.

> ℹ️ Continuous Delivery ≠ Continuous Deployment. Delivery = always ready to deploy (human gate). Deployment = automatically deployed to production (no human gate). Automotive safety-critical systems will always require Continuous Delivery (not Deployment) with explicit human safety sign-off.

4.4 Pipeline Design Principles

4.4.1 Fast Feedback First

Structure pipeline stages so the fastest feedback comes first. Unit tests before integration tests before E2E tests. Static analysis before dynamic analysis. This principle, called the 'fail-fast' strategy, minimizes wasted computation and maximizes developer feedback speed.

4.4.2 Parallelism

Leverage parallel pipeline execution to reduce wall-clock time. Tests can often be split across multiple agents simultaneously. For a typical automotive SW build, splitting MISRA-C checking, unit tests, and integration tests across parallel Jenkins agents reduces pipeline time from 45+ minutes to under 20 minutes.

4.4.3 Pipeline as Code

Store pipeline definitions as code in the source repository (Jenkinsfile, .gitlab-ci.yml, .github/workflows/*.yaml). This ensures pipeline definitions are versioned, reviewed, and tested alongside the application code.

4.4.4 Immutable Artifacts

Build the artifact once and promote it through environments. Never rebuild from source for staging or production deployments — the artifact validated in testing is identical to what goes to production. JFrog Artifactory's promotion mechanism enforces this.

4.5 Trunk-Based Development vs. Feature Branching

Two primary branching strategies exist for CI/CD:

- **Trunk-Based Development (TBD): All developers commit directly to main/trunk (or short-lived feature branches of ≤ 2 days). Requires feature flags for in-progress work. Maximizes CI effectiveness. Adopted by Google, Facebook, Netflix.**
- **GitFlow: Long-lived feature, develop, release, and hotfix branches. More suitable for teams shipping versioned releases (e.g., automotive ECU software with discrete software versions for OEM validation).**

For automotive software, a hybrid is often optimal: trunk-based development for individual components within a module, with GitFlow-style release branches for final ECU software composition (SWP/SWC integration) before OEM delivery.

4.6 Quality Gates and Definition of Done

Quality gates are automated checkpoints that must pass before a pipeline can proceed to the next stage. They encode the organisation's Definition of Done (DoD) into the pipeline. Example quality gate thresholds:

Gate	Threshold	Tool
Unit Test Pass Rate	100%	GTest / JUnit
Code Coverage (line)	>= 85% (ASIL-B), 100% MC/DC (ASIL-D)	LCOV / Bullseye
MISRA-C Violations	0 required, deviations documented	PC-lint / QAC
SonarQube Rating	A (no new code smells)	SonarQube
Security Vulnerabilities	0 Critical/High (unaccepted)	JFrog Xray
Build Warnings	0 new compiler warnings	GCC / MSVC

4.7 CI/CD Platform Comparison: Jenkins vs. GitHub Actions vs. GitLab CI

While Jenkins remains dominant in automotive and regulated industries due to its flexibility and on-premise deployment model, GitHub Actions and GitLab CI have achieved broad adoption for cloud-native and open-source projects. In 2026, many organisations use all three in parallel - Jenkins for automotive/embedded pipelines, GitHub Actions for open-source components, GitLab CI for DevSecOps integration. Understanding the trade-offs is essential for practitioners choosing or migrating CI/CD tooling.

```
# GitHub Actions: equivalent automotive build pipeline
name: ECU Software CI
on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:
  build-and-verify:
    runs-on: [self-hosted, automotive-linux]  # On-prem runner
    container:
      image: registry.company.com/automotive-builder:1.2.0
    steps:
      - uses: actions/checkout@v4

      - name: CMake Build
        run: |
          mkdir build && cd build
          cmake .. -DCMAKE_BUILD_TYPE=Release
          make -j$(nproc)

      - name: MISRA-C Check
        run: find src/ -name '*.c' | xargs pclint misra_c_2012.lnt

      - name: Unit Tests + Coverage
        run: cd build && ctest --output-on-failure

      - name: Upload to JFrog
        env:
          JFROG_TOKEN: ${{ secrets.JFROG_API_TOKEN }}
        run: |
                        jfrog  rt  upload  build/*.hex  automotive-dev-local/my-ecu/$
{{ github.run_number }}/

      - name: Upload test results
        uses: actions/upload-artifact@v4
        with:
          name: test-results
          path: build/test-results/

push:
```

```
branches: [main, develop]
pull_request:
branches: [main]
jobs:
build-and-verify:
runs-on: [self-hosted, automotive-linux]
# On-prem runner
container:
image: registry.company.com/automotive-builder:1.2.0
steps:
- uses: actions/checkout@v4
- name: CMake Build
run: |
mkdir build&&cd build
cmake .. -DCMAKE_BUILD_TYPE=Release
make -j$(nproc)
- name: MISRA-C Check
run: find src/ -name'*.c' | xargs pclint misra_c_2012.lnt
- name: Unit Tests + Coverage
run: cd build&&ctest --output-on-failure
- name: Upload to JFrog
env:
JFROG_TOKEN: ${{ secrets.JFROG_API_TOKEN }}
run: |
jfrog rt upload build/*.hex automotive-dev-local/my-ecu/${{ github.run_number }}/
- name: Upload test results
uses: actions/upload-artifact@v4
with:
name: test-results
path: build/test-results/
```

Key differences: Jenkins requires manual plugin installation and controller maintenance (self-hosted only); GitHub Actions provides managed runners with YAML-native workflow syntax and deep GitHub integration; GitLab CI offers the tightest source-code-to-pipeline integration with built-in container registry, security scanning, and DAST. For safety-critical automotive software, Jenkins or GitLab CI self-hosted are preferred - cloud-managed runners introduce supply chain risks incompatible with ISO 26262 / UN R155 requirements without additional mitigations.

Chapter 5: Git — Distributed Version Control

5.1 Git Architecture

Git is a distributed version control system created by Linus Torvalds in 2005 for managing the Linux kernel source. Unlike centralized VCS (SVN, Perforce), every Git clone is a complete repository with full history. This enables offline operation, fast local operations, and multiple workflows.

5.1.1 Git Object Model

Git stores content as four types of objects in a content-addressed store (.git/objects):

- **Blob — Stores file content (no filename or directory info). SHA-1/SHA-256 hashed.**
- **Tree — Stores a directory listing: pointers to blobs (files) and other trees (subdirectories).**
- **Commit — Points to a tree (root of working tree at that point) + parent commits + metadata.**
- **Tag — Named pointer to a commit (annotated tags also store tagger identity and message).**

```
# Git internals: examining the object store
git cat-file -t HEAD           # shows 'commit'
git cat-file -p HEAD           # shows commit object
git log --graph --oneline --all  # visualize DAG
```

5.2 Git Workflows

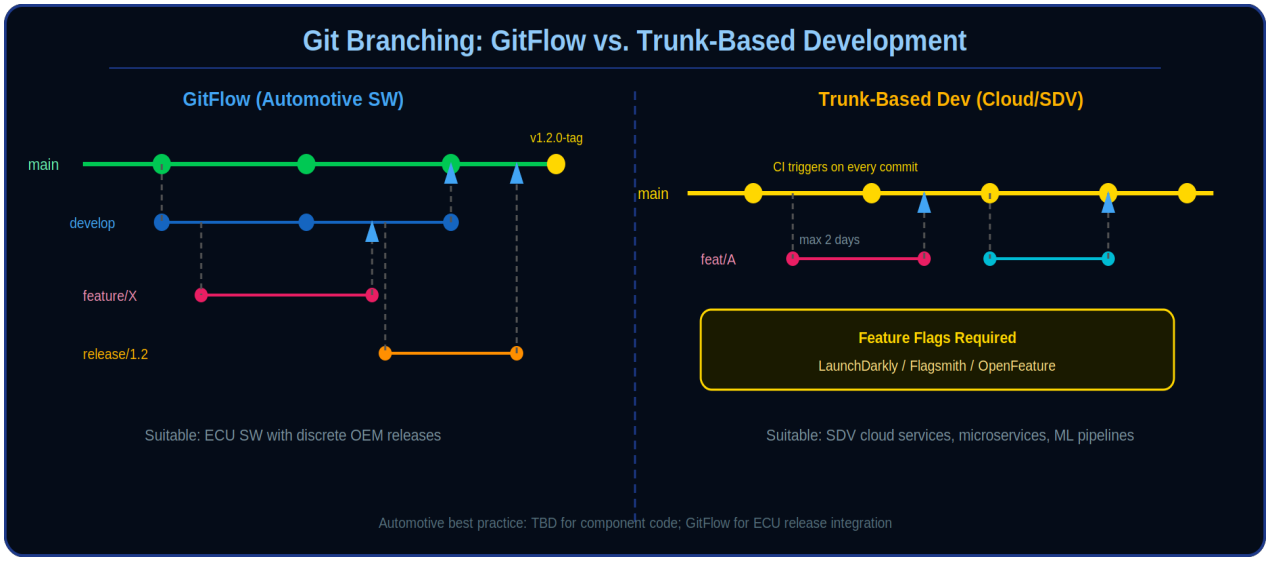

Figure 5.1 — GitFlow Branching Strategy with Feature, Release, and Hotfix Branches

5.2.1 GitFlow

Vincent Driessen's GitFlow model (2010) defines a strict branching structure suited to versioned software releases. It distinguishes long-lived (main, develop) and short-lived (feature/*, release/*, hotfix/*) branches. Ideal for automotive software shipping ECU software packages to OEMs on defined cadences.

- **main: Always reflects production-ready code. Tagged with version numbers (v1.0, v2.3.1).**

- develop: Integration branch. Feature branches merge here via Pull Requests.
- feature/*: One branch per user story or ticket. Created from develop, merged back to develop.
- release/*: Branched from develop when the release is feature-complete. Only bug fixes allowed.
- hotfix/*: Emergency fixes branched directly from main. Merged to both main and develop.

5.2.2 Trunk-Based Development

Short-lived feature branches (max 2 days) or direct commits to trunk/main. Feature flags hide incomplete features. Requires comprehensive automated testing to maintain trunk stability. Preferred by high-performing DevOps organizations for faster CI cycle times.

5.3 Advanced Git Techniques

5.3.1 Git Hooks

Git hooks are scripts that execute at specific points in the Git workflow. Client-side hooks enforce local policies; server-side hooks enforce repository policies.

```
# pre-commit hook: run MISRA check before commit
#!/bin/bash
files=$(git diff --cached --name-only --diff-filter=ACM | grep '\.c$')
if [ -n "$files" ]; then
    pclint $files
    if [ $? -ne 0 ]; then
        echo "MISRA-C check failed. Commit blocked."
        exit 1
    fi
fi
```

5.3.2 Git LFS (Large File Storage)

Git LFS replaces large files (binaries, ARXML, calibration data, images) in the repository with lightweight pointer files, while storing actual file contents on a remote server (GitHub LFS, GitLab LFS, or Bitbucket LFS). Critical for automotive SW repositories containing ARXML configuration files, .hex files, and MATLAB/Simulink models.

```
# Configure Git LFS for automotive file types
git lfs install
git lfs track "*.arxml"
git lfs track "*.hex"
git lfs track "*.elf"
git lfs track "*.slx"  # Simulink models
git lfs track "*.mdl"
git add .gitattributes
```

5.3.3 Submodules and Subtrees

Git submodules allow embedding one Git repository as a subdirectory of another while maintaining independent history. In automotive SW, this is used to manage AUTOSAR BSW components (e.g., MCAL from silicon vendors) as submodules within the application repository. Git subtrees are an alternative with simpler workflow but interleaved histories.

5.4 Git Best Practices for Automotive SW Teams

5.4.1 Commit Message Conventions

```
# Conventional Commits format (enforced by commit-lint)
```

```
feat(dcm): add UDS 0x27 security access handler
fix(dem): correct DTC freeze frame storage on overflow
docs(readme): update AUTOSAR version compatibility table
chore(ci): upgrade Jenkins pipeline to LTS 2.426
refactor(rte): extract Rte_Invalidate macros to common header

# Format: type(scope): description
# Breaking changes: add BREAKING CHANGE: footer
```

5.4.2 Branch Protection Rules

- **Require pull request reviews (minimum 2 reviewers for safety-critical code)**
- Require status checks: CI build + MISRA check must pass before merge
- Require up-to-date branches: must rebase on latest main before merge
- Restrictforce-push: never allow force-push to main or develop
- Require signed commits: GPG-signed commits for auditability (ISO 26262 traceability)
- Linear history: use squash merge or rebase to maintain clean history

Chapter 6: Jenkins — The Automation Server

6.1 Jenkins Architecture

Jenkins is an open-source automation server written in Java. Originally created as Hudson by Kohsuke Kawaguchi at Sun Microsystems in 2004, it was forked and renamed Jenkins in January 2011 following an Oracle acquisition dispute. It is the most widely deployed CI/CD automation server globally, with over 300,000 active installations. Jenkins operates on a controller-agent (formerly master-slave) model where the controller manages job scheduling, web UI, and configuration, while agents execute the build tasks.

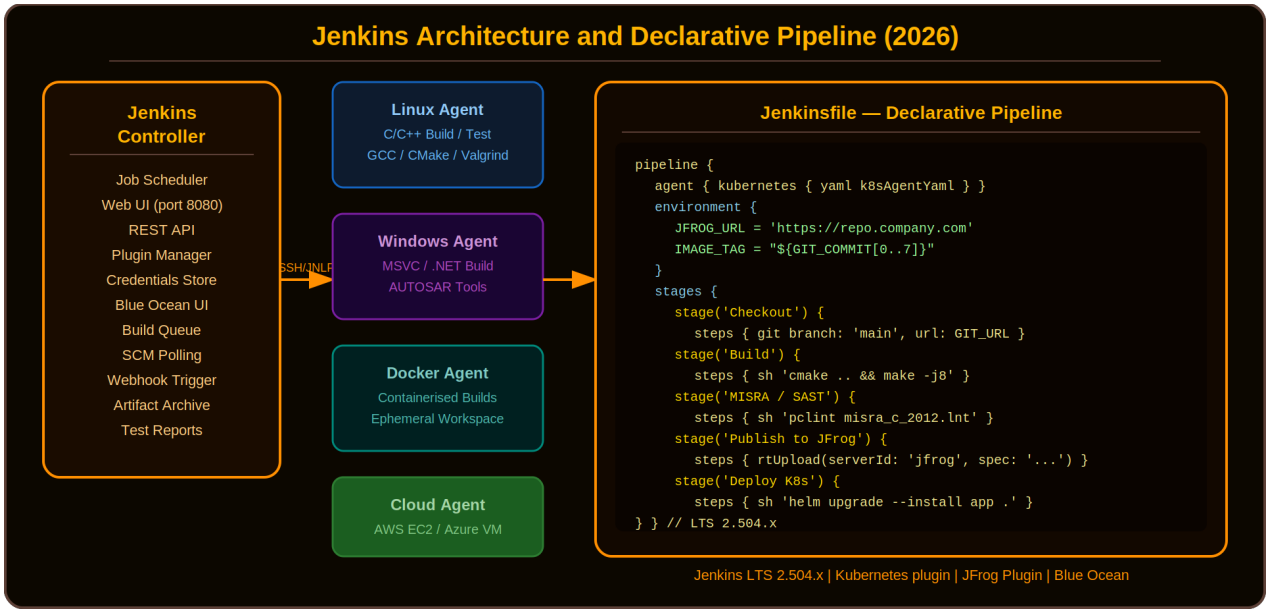

Figure 6.1 — Jenkins Controller–Agent Architecture with Declarative Pipeline Code Example

6.2 Jenkins Installation and Security

6.2.1 Production-Grade Installation

```
# Docker Compose: Jenkins with persistent storage version: '3.8' services:
jenkins:
image: jenkins/jenkins:lts-jdk17
user: root
ports:
-"8080:8080"-"50000:50000"# JNLP agent port
volumes:
- jenkins_data:/var/jenkins_home
- /var/run/docker.sock:/var/run/docker.sock
# SECURITY NOTE: Mounting docker.sock grants root-equivalent host access.
# Hardened alternatives: Kaniko (rootless image builds) or Docker-in-Docker (DinD)
# with --privileged flag scoped to dedicated build pods only.
environment:
- JENKINS_OPTS=--prefix=/jenkins
- JAVA_OPTS=-Xmx2g -XX:+UseG1GC volumes:
jenkins_data:
```

6.2.2 Security Hardening

- **Enable Jenkins Security Realm: LDAP / Active Directory integration**
- Authorization: Matrix-based security or Role-Based Access Control (RBAC) plugin
- CSRF Protection: Enable Crumb issuer (default in Jenkins 2.x)
- Agent-to-Controller Security: Restrict agent file system and class loading

- Secret Management: Use Jenkins Credentials Store, never hardcode in Jenkinsfile
- Network: Place Jenkins behind reverse proxy (Nginx/Apache) with TLS
- Audit: Enable Audit Trail Plugin; log all build and configuration changes

6.3 Declarative vs. Scripted Pipelines

Jenkins offers two Pipeline-as-Code syntaxes. Declarative Pipeline (introduced 2017) provides a more structured, opinionated syntax with built-in error handling and is the recommended approach. Scripted Pipeline uses full Groovy syntax for maximum flexibility but is harder to read and maintain.

6.3.1 Comprehensive Declarative Pipeline

```
pipeline {
  agent none
  options {
    buildDiscarder(logRotator(numToKeepStr: '10'))
    timeout(time: 60, unit: 'MINUTES')
    disableConcurrentBuilds()
    ansiColor('xterm')
  }
  environment {
    JFROG_URL   = credentials('jfrog-server-url')
    JFROG_CREDS = credentials('jfrog-credentials')
    SONAR_TOKEN = credentials('sonarqube-token')
    APP_VERSION = "${env.BUILD_NUMBER}"
  }
  stages {
    stage('Checkout') {
      agent { label 'linux-build-agent' }
      steps {
        checkout scm
        sh 'git log --oneline -5'
      }
    }
    stage('Parallel: Build + Lint') {
      parallel {
        stage('CMake Build') {
          agent { label 'linux-build-agent' }
          steps {
                sh 'mkdir -p build && cmake .. -DCMAKE_BUILD_TYPE=Release && make -j$
(nproc)'
          }
        }
        stage('MISRA-C Check') {
          agent { label 'misra-agent' }
          steps { sh 'pclint +v src/**/*.c -format=lint -max_errors=0' }
        }
      }
    }
    stage('Unit Tests + Coverage') {
      agent { label 'linux-build-agent' }
      steps {
        sh 'cd build && ctest --output-on-failure'
        sh 'gcov src/*.c && lcov --capture --directory . --output-file coverage.info'
      }
      post {
        always {
          junit 'build/test-results/**/*.xml'
          publishHTML([reportDir: 'coverage-report', reportName: 'Coverage'])
        }
      }
    }
    stage('Publish to JFrog') {
      agent { label 'linux-build-agent' }
      steps {
        rtUpload(serverId: 'jfrog-artifactory',
           spec: '{"files":[{"pattern":"build/*.hex","target":"automotive-local/my-ecu/$
{APP_VERSION}/"}]}')
        rtPublishBuildInfo(serverId: 'jfrog-artifactory')
```

```
      }
    }
  }
  post {
    failure { emailext subject: 'BUILD FAILED: ${JOB_NAME} #${BUILD_NUMBER}',
                        body: '${BUILD_LOG}', to: 'team@company.com' }
      success { slackSend color: 'good', message: "Build ${JOB_NAME} #${BUILD_NUMBER}
succeeded" }
    always  { cleanWs() }
  }
}
#${BUILD_NUMBER}',  body:  '${BUILD_LOG}',  to:  'team@company.com'  }          success
{ slackSend color: 'good', message:"Build ${JOB_NAME}
#${BUILD_NUMBER} succeeded"}     always   { cleanWs() }   } }
```

6.4 Jenkins Shared Libraries

Shared Libraries allow reusing Groovy code across multiple Jenkinsfiles in the organisation. This is critical for enforcing standards — if every team's Jenkinsfile calls a shared library's standardMisraCheck() function, MISRA compliance is guaranteed across all pipelines.

```
// vars/standardMisraCheck.groovy (Shared Library function)
def call(String srcDir = 'src') {
  sh"""#!/bin/bash
    pclint +v ${srcDir}/**/*.c \\
           -MISRA2012:required \\
           -format=lint \\
           -max_errors=0 || exit 1"""
}
// Usage in any Jenkinsfile:
@Library('automotive-shared-lib') _
pipeline {
  stages { stage('MISRA') { steps { standardMisraCheck('src/appl') } } }
}
```

6.5 Jenkins Plugins Ecosystem

Jenkins has over 1,900 plugins (plugins.jenkins.io, June 2026). For automotive SW CI/CD, the most important are:

Plugin	Purpose	Automotive Relevance
JFrog Artifactory	Artifact upload/download/promote	Store .hex, .elf, ARXML packages
Git Parameter	Parameterized builds with git refs	Release builds from specific tags
Docker Pipeline	Docker build/run within pipeline	Containerized build toolchains
SonarQube Scanner	Static code analysis integration	Code quality gates
Matrix Authorization Strategy	Fine-grained RBAC	Separation of dev/safety roles
Email Extension (edx)	Rich build notifications	Build failure alerts to OEM teams
Timestamper	Add timestamps to console logs	Audit trail for compliance
Pipeline Stage View	Visual pipeline progress	Stage timing analysis
Credentials Binding	Inject secrets into pipeline env	Secure key management

Chapter 7: JFrog — Universal Binary Repository Manager

7.1 JFrog Platform Overview

JFrog is a DevOps platform company that provides tools for binary management, security scanning, distribution, and pipeline orchestration. The flagship product, JFrog Artifactory, is the universal binary repository manager that supports virtually every package format used in software development. JFrog's broader platform — including Xray (security), Pipelines (CI/CD), Distribution, and Connect (IoT/edge) — forms a complete DevSecOps toolchain.

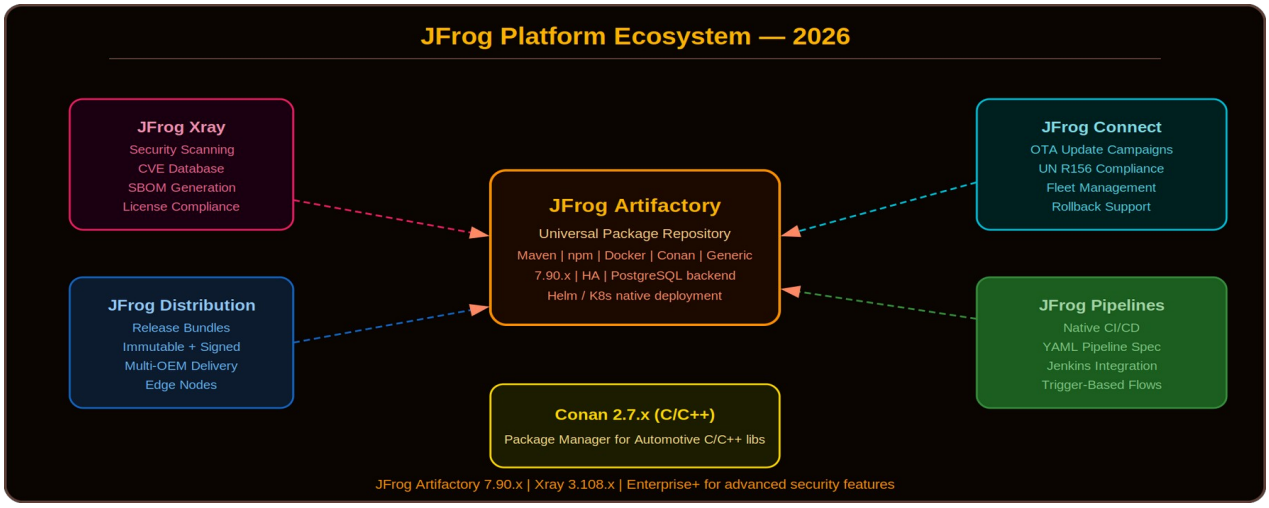

Figure 7.1 — JFrog Platform Architecture: Artifactory at the Center with Xray, Pipelines, Distribution, and Connect

7.2 JFrog Artifactory Deep Dive

7.2.1 Repository Types

- **Local Repository: Stores artifacts produced internally (build artifacts, proprietary packages). Source of truth for your organization's binaries.**
- **Remote Repository (Proxy): Caches artifacts from external repositories (npmjs.org, Maven Central, PyPI, Docker Hub). Improves build speed and reliability. Provides audit and control over third-party dependencies.**
- **Virtual Repository: Aggregates local and remote repos under a single URL. Clients interact with one URL; Artifactory resolves from underlying repos in priority order. Simplifies build tool configuration.**

7.2.2 Package Formats Supported

Artifactory supports: Docker, Helm, Maven, Gradle, npm, PyPI, Conan (C/C++), NuGet (.NET), RubyGems, Cargo (Rust), Go modules, Debian/Ubuntu, RPM, Alpine, Conda (Python data science), Terraform, CloudFormation, Generic (any file type). The Conan support is particularly important for C/C++ automotive software dependency management.

7.2.3 Conan Repository for C/C++

```
# conanfile.txt for AUTOSAR BSW component
[requires]
boost/1.82.0
fmt/10.1.1
openssl/3.1.2
autosar-mcal/4.4.0@supplier/stable
```

```
[generators]
cmake

# Configure Artifactory as Conan remote
conan remote add mycompany https://artifactory.company.com/artifactory/api/conan/conan-local
conan remote list
```

7.2.4 Build Integration and Promotion

Artifactory captures build information — which artifacts were produced, from which source, with which dependencies — creating a complete Software Bill of Materials (SBOM) and enabling traceability from binary to source commit. Build promotion moves artifacts between repositories (dev-local → staging-local → release-local) with metadata enrichment.

```
// Jenkinsfile: JFrog Artifactory integration
stage('Promote to Staging') {
  steps {
    rtPromote(
      serverId: 'jfrog-artifactory',
      sourceRepo: 'automotive-dev-local',
      targetRepo: 'automotive-staging-local',
      copy: false,  // move (promote), not copy
      properties: "release.status=staging;tested.by=${env.BUILD_NUMBER}")
  }
}
```

7.3 JFrog Xray — Security and Compliance

JFrog Xray is a universal software composition analysis (SCA) and security scanning tool deeply integrated with Artifactory. It performs recursive deep scanning — scanning not just the top-level package but all transitive dependencies — using the JFrog Security Research database, NVD (NIST), and OSS Index.

7.3.1 Xray Capabilities

- **CVE Scanning: Deep recursive dependency scanning with CVSS scoring and fix availability**
- License Compliance: Automated detection and enforcement of acceptable open-source licenses
- Operational Risk: End-of-life detection, deprecation warnings, low community activity flags
- SBOM Generation: Export SPDX 2.3/3.0 or CycloneDX 1.6 SBOMs for regulatory compliance
- Policy Enforcement: Automatically block promotion or download of artifacts violating policies
- JAS (JFrog Advanced Security): Secret detection, SAST, Infrastructure-as-Code scanning

7.3.2 Security Policy Example

```
// Xray Policy: Block Critical CVEs in Production
{
  "name": "automotive-critical-cve-policy",
  "type": "security",
  "rules": [{
    "name": "block-critical",
    "priority": 1,
    "criteria": {
      "min_severity": "Critical",
      "fix_version_dependant": true
    },
    "actions": {
      "block_release_bundle_distribution": true,
      "fail_build": true,
      "notify_email": ["security-team@company.com"]
    }
  }]
}
```

7.4 JFrog Distribution and Release Bundles

JFrog Distribution enables secure, efficient software distribution to edge nodes globally. A Release Bundle is a curated, immutable, cryptographically signed collection of artifacts representing a software release. This maps directly to automotive use cases: a Release Bundle for ECU Software Package v2.3.1 contains .hex files, AUTOSAR configurations, and metadata.

- **Immutable: Once created, a Release Bundle cannot be modified. Any change requires a new version.**
- Signed: Cryptographic signatures ensure integrity and authenticity — critical for UN R155 cybersecurity compliance.
- Traceable: Full lineage from source commit → build → test results → distribution → edge ECU.
- Atomic distribution: All-or-nothing distribution to each edge node; no partial updates.

7.5 JFrog Connect — IoT and Automotive OTA

JFrog Connect extends the JFrog platform to IoT and edge device management. For automotive applications, it enables software update campaigns for vehicle ECUs — analogous to the AUTOSAR UCM (Update and Configuration Management) master functionality but with JFrog's enterprise-grade platform features.

> ℹ️ JFrog Connect + JFrog Artifactory + JFrog Xray provides the complete OTA pipeline: secure artifact storage (Artifactory) → security scan before distribution (Xray) → signed update campaign deployment (Connect). This architecture aligns with UN R156 (Software Updates) requirements.

Chapter 8: Platform Engineering and Internal Developer Platforms

8.1 Platform Engineering: The New Paradigm (2026)

Platform Engineering is the discipline of designing and building internal developer platforms (IDPs) — self-service platforms that enable cross-functional teams (frontend, backend, ML, embedded, QA, security engineers) to provision infrastructure, deploy applications, and manage the entire software delivery lifecycle with minimal toil. Unlike SRE (Site Reliability Engineering), which focuses on reliability, Platform Engineering focuses on developer experience, standardization, and reducing cognitive load.

The CNCF Platform Engineering White Paper (2024–2025) identifies Platform Engineering as distinct from DevOps: while DevOps breaks silos between Dev and Ops, Platform Engineering goes further by building abstraction layers that hide infrastructure complexity and enable self-service. This is critical for scaled engineering organizations (50–5000+ engineers) where DevOps responsibility scales exponentially.

8.2 Internal Developer Platform (IDP) Architecture

An IDP orchestrates and abstracts the underlying tools we've covered: Git repositories, Jenkins/GitLab CI/GitHub Actions for CI/CD, JFrog Artifactory for artifact management, Kubernetes for container orchestration, and observability stacks (Prometheus/Grafana). The key insight: developers interact with the IDP's self-service portal, not directly with Jenkins or Kubernetes.

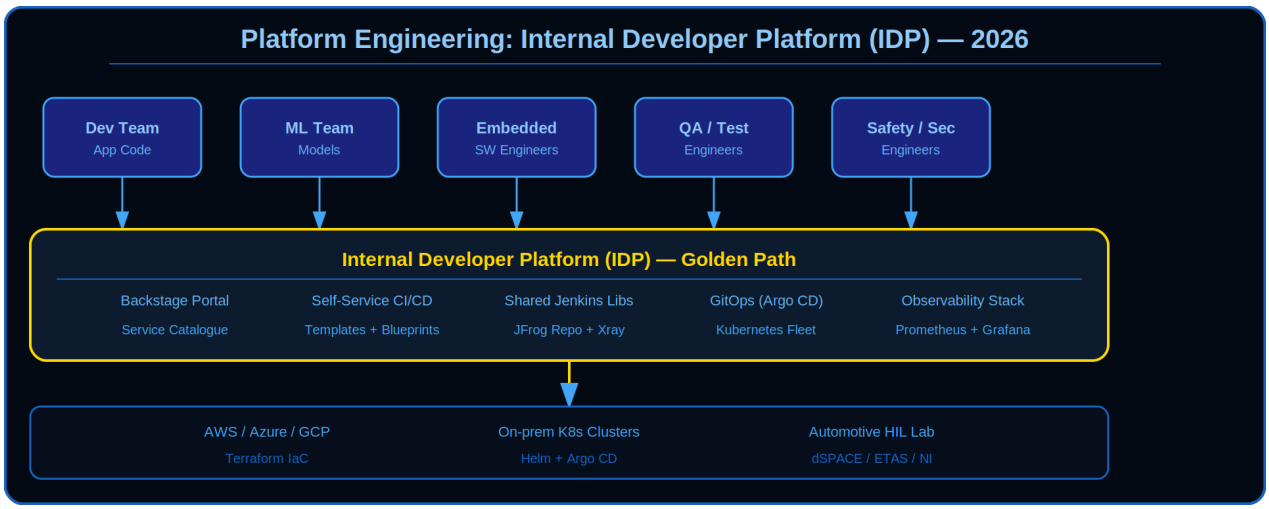

Figure 8.1 — Platform Engineering: IDP Orchestrating Git, Jenkins, JFrog, K8s, and Observability for Multi-Discipline Teams

8.3 Core IDP Components

8.3.1 Service Catalogue (Backstage)

Spotify's Backstage is the de facto open-source service catalogue and IDP. It provides: a registry of all services (with ownership, docs, on-call info), templated service creation (scaffolding), and plugin architecture for integrating external tools. Teams self-serve by creating a new service from a template rather than manually provisioning infrastructure.

8.3.2 CI/CD Self-Service

The IDP exposes CI/CD as self-service: developers define a Backstage template for a new microservice; the template automatically creates a Git repo, provisions a CI/CD pipeline in Jenkins (or Argo Workflows), configures automated testing, and sets up deployment to Kubernetes. No manual intervention required.

8.3.3 Shared Jenkins Libraries&Templates

Centralized, versioned Jenkins Shared Libraries encode best practices: standard build stages, security scanning (SAST, Xray scans), code quality gates, artifact promotion logic, and deployment patterns. All teams use the same, audit-tested library, reducing drift and duplicated work.

```
// Jenkins Shared Library (library.groovy) — Centralized CI/CD standards
@Library('company-devops-library@main') _

pipeline {
  agent any
  stages {
    stage('Build & Test') {
      steps {
        script {
          buildAndTest(language: 'cpp', standards: ['MISRA-C', 'cppcheck'])
        }
      }
    }
    stage('Security Scan') {
      steps {
        script {
          securityScan(tools: ['jfrog-xray', 'sonarqube', 'snyk'])
        }
      }
    }
  }
}
```

8.4 GitOps and Infrastructure as Code (IaC)

Platform Engineering mandates that infrastructure, deployment configurations, and observability rules are defined as code and stored in Git. Tools like ArgoCD (continuous deployment for Kubernetes), Flux (continuous orchestration), and Terraform (infrastructure provisioning) enable this. A golden principle: if it's not in Git, it doesn't exist.

8.4.1 ArgoCD vs FluxCD — Selecting the Right GitOps Engine

Both Argo CD and Flux are CNCF Graduated projects that implement the GitOps pull-model: agents running inside the cluster continuously reconcile desired state stored in Git with live cluster state. They differ significantly in architecture, operational philosophy, and built-in capabilities. The choice matters: migrating between them in a large platform is a significant effort.

Dimension	Argo CD 2.13.x	Flux CD 2.4.x
Architecture	Centralised API server + UI + Dex (SSO)	Modular controllers: source, kustomize, helm, image, notification
Web UI	Built-in rich dashboard (real-time sync graph)	Optional via Weave GitOps Enterprise; CLI-first by default
Multi-tenancy	AppProject CRDs with RBAC and namespace isolation	Per-GitRepository tenancy; Flux Kustomization scoping
App definition CRD	Application / ApplicationSet	Kustomization / HelmRelease / OCIRepository
Drift detection	Continuous; visual real-time diff in UI	Event-driven reconciliation

		(configurable interval, default 10 min)
Image auto-update	Argo CD Image Updater (separate add-on)	Built-in Image Automation Controller
Notification	Notification Controller (separate add-on)	Built-in Notification Controller (Slack, Teams, PagerDuty)
Secret management	Integrates with Vault Agent, Sealed Secrets, External Secrets	Native SOPS decryption; External Secrets Operator integration
CNCF Status	Graduated — April 2020	Graduated — November 2021
Best suited for	App-centric GitOps; platform teams needing UI visibility	Infrastructure GitOps; operator-pattern; image automation workflows

When to choose Argo CD: platform teams that need a visual dashboard for application deployment status across many services will find Argo CD's real-time UI indispensable. ApplicationSet enables templated multi-cluster deployment from a single configuration, making it the preferred choice for app-centric GitOps at OEM scale.

- Rich web UI for visualising sync status, resource health, and deployment history
- ApplicationSet controller: generate hundreds of Application CRDs from a single template (e.g., one per environment × service combination)
- Project-level RBAC: isolate dev, staging, and production environments with AppProject boundaries and source repository allow-lists
- GitOps Engine library: embeddable reconciliation logic for building custom controllers

When to choose Flux: teams following an operator-pattern philosophy — where everything in the cluster is managed by a Kubernetes controller — will find Flux's modular design more aligned. The built-in Image Automation Controller automatically commits new image tags to Git without additional tooling, making it the stronger choice for automated container lifecycle management.

- Image Automation Controller: automatically updates image tags in Git when new images are pushed to the registry — closes the GitOps loop without external CI
- SOPS native decryption: secrets can be encrypted in Git using Mozilla SOPS and decrypted transparently at runtime without a sidecar
- Smaller resource footprint: Flux controllers are individually scoped and consume less memory than a full Argo CD deployment at idle
- HelmRelease CRD: declarative Helm chart lifecycle with drift detection and automatic upgrade/rollback on values or chart version changes

```
# Argo CD ApplicationSet — multi-service production rollout
apiVersion: argoproj.io/v1alpha1
kind: ApplicationSet
metadata:
  name: microservices-prod   namespace: argocd
spec:
  generators:

   - list:
       elements:

         - service: order-service             team: commerce
          - service: payment-service             team: finance
          - service: inventory-service             team: commerce   template:
    metadata:
      name: "{{service}}-production"       annotations:
            notifications.argoproj.io/subscribe.on-sync-failed.slack:  "{{team}}-alerts"
spec:
     project: production       source:
         repoURL: https://github.com/myorg/k8s-manifests           targetRevision: HEAD
path: "production/{{service}}"       destination:
           server:  https://kubernetes.default.svc             namespace:  "{{service}}"
```

```
syncPolicy:
        automated:
          prune: true           selfHeal: true         syncOptions:

         - CreateNamespace=true
          - ServerSideApply=true
```

⚠️ Production recommendation: Large platform teams often deploy both. Argo CD manages application workloads (microservices, LLM apps) where the UI adds value; Flux manages cluster infrastructure (Prometheus, Cert-Manager, Vault, CNI) where the operator pattern and SOPS-encrypted secrets provide better separation of concerns. This dual-GitOps pattern is documented in the CNCF GitOps Working Group best practices (2024).

8.5 Observability Stack in the IDP

Every service created through the IDP automatically gets: pre-configured Prometheus scrape endpoints, Grafana dashboards (templated), centralized logging (ELK Stack / Loki), and distributed tracing (Jaeger / OpenTelemetry). This 'observability by default' removes the friction of manually wiring monitoring.

8.6 Platform Engineering in Automotive

In automotive, an IDP accelerates ECU software development by standardizing: CI/CD pipelines for embedded C/C++ (with MISRA-C, ISO 26262 compliance checking), automated AUTOSAR configuration generation, HIL/SIL/PIL testing orchestration, and OTA campaign management. Teams self-serve AUTOSAR SWC scaffolding, automated traceability to requirements (DOORS/JIRA), and safety evidence generation (functional safety templates).

ℹ️ Automotive Platform Engineering Maturity: Level 1 (2024–2025): Basic shared Jenkins libraries and Kubernetes templates. Level 2 (2026+): Full Backstage IDP with AUTOSAR/safety-aware service templates, automated traceability, and OTA pipeline orchestration. Level 3 (2027+): AI-augmented platforms using GenAI for architecture recommendations, test generation, and failure prediction.

Chapter 9: Automotive Software Development Context

9.1 The Software-Defined Vehicle Revolution

Modern vehicles contain 100–150 ECUs running 100+ million lines of software code. The transition to Software-Defined Vehicles (SDVs) — where vehicle functions are delivered and updated through software rather than hardware changes — is fundamentally transforming automotive software development. OEMs like Tesla, GM, BMW, and Volkswagen are adopting centralized compute architectures with high-performance computers (HPCs) running Linux/AUTOSAR Adaptive, replacing distributed ECU networks.

This transformation demands a corresponding shift in development practices: from long waterfall cycles (2-3 years for traditional ECU SW) to continuous delivery with OTA updates enabling monthly or even weekly feature releases. The challenge is doing this while meeting the functional safety and regulatory requirements that automotive electronics demands.

9.2 AUTOSAR Overview

9.2.1 AUTOSAR Classic Platform

AUTOSAR Classic (CP) defines a standardised software architecture for deeply embedded, hard real-time automotive control units (microcontrollers without OS abstractions). Key layers: MCAL (Microcontroller Abstraction Layer), ECU Abstraction Layer, Service Layer, and Application Layer. BSW (Basic Software) is the standardised middleware layer managed by the AUTOSAR consortium.

- **Tools: Vector DaVinci Developer/Configurator, EB Tresos Studio, ISOLAR-A (Arccore/Arctic Core)**
- Communication: COM, PDU Router, CAN/LIN/Ethernet transport protocol stacks
- Diagnostics: DCM (Diagnostic Communication Manager), DEM (Diagnostic Event Manager), UDS on CAN/IP
- OS: OSEK-compatible real-time operating system (EB tresos AutoCore OS, Vector MICROSAR OS)

9.2.2 AUTOSAR Adaptive Platform

AUTOSAR Adaptive (AP) targets high-performance computing units (application processors, SoCs) running POSIX-compliant operating systems (e.g., QNX, Linux). It enables dynamic service deployment, Ethernet-based communication (SOME/IP, DDS), and is designed for ADAS, infotainment, and telematics domains.

- **ARA (Adaptive AUTOSAR Runtime for Applications): The middleware API**
- Execution Management: aracom, persistent storage, crypto stack
- Communication Management: SOME/IP, DDS, RPC over Ethernet
- Update and Configuration Management (UCM): Native OTA update mechanism

9.3 Regulatory Framework

9.3.1 ISO 26262 — Functional Safety

ISO 26262 is the functional safety standard for road vehicles, defining requirements for hazard analysis, safety goals, safety concepts, and verification/validation at ASIL (Automotive Safety Integrity Level) A through D. CI/CD pipelines for safety-critical software must automate many ISO 26262 process requirements.

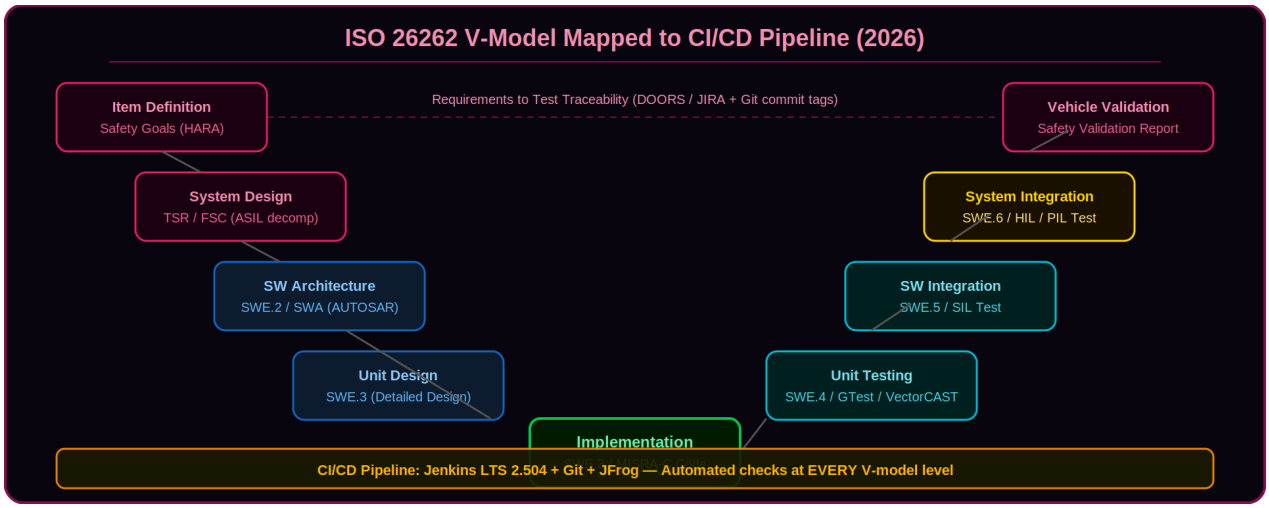

Figure 9.1 — ISO 26262 V-Model Mapped to CI/CD Pipeline Stages and Automated Checks

9.3.2 ASPICE — Process Capability

ASPICE (Automotive SPICE — Software Process Improvement and Capability dEtermination) is the de facto software process assessment framework for automotive suppliers. OEMs require suppliers to achieve ASPICE Capability Level 2 or 3. CI/CD automation directly supports multiple ASPICE process attributes: Work Product Management (PA 2.1), Process Definition (PA 3.1), and Process Performance (PA 2.2).

ASPICE Process	CI/CD Automation Support	Tools
SWE.1 (Req Elicitation)	Requirement IDs in commit messages; JIRA traceability	DOORS, JIRA, Git commit hooks
SWE.2 (SW Architecture)	ARXML version control; architecture diff checks	Git LFS, DaVinci, ISOLAR
SWE.3 (Detailed Design)	Code review enforcement via PR gates	GitLab/GitHub PR + Jenkins
SWE.4 (Unit Verification)	Automated unit test execution; coverage reporting	GTest/VectorCAST + Jenkins + LCOV
SWE.5 (SW Integration)	Automated SIL integration test runs	VectorCAST/SIL, Jenkins
SWE.6 (SW Qualification)	HIL/PIL test automation; test report archiving	dSPACE HIL + JFrog artifact store

9.3.3 UN R155/R156 — Cybersecurity and OTA

UN Regulation 155 (cybersecurity) and UN Regulation 156 (software updates) are UNECE WP.29 regulations mandatory for vehicle type approval in the EU, UK, Japan, Korea, and other regions. They applied to new vehicle types from July 2022 and to all new vehicles from July 2024. They impose binding process requirements on OEMs and their entire supply chain.

9.3.4 ISO/SAE 21434 — Cybersecurity Engineering

ISO/SAE 21434:2021 (Road Vehicles - Cybersecurity Engineering) is the international standard specifying the engineering processes to achieve and maintain cybersecurity throughout the vehicle lifecycle. Where UN R155 mandates the WHAT (CSMS existence and type approval), ISO 21434 defines the HOW (specific activities, work products, and evidence required at each development phase).

Key ISO 21434 activities mapped to CI/CD: TARA (Threat Analysis and Risk Assessment) produces the cybersecurity goals that must be traced to software requirements in the same way as ISO 26262 safety goals. Cybersecurity concept and design reviews are gated by pull-request policies. Vulnerability

analysis (CVSS scoring via JFrog Xray) runs in every pipeline execution. Penetration testing results are stored as build artefacts in JFrog Artifactory for supply chain evidence. Security regression tests run alongside functional tests in SIL/HIL environments.

```
# JFrog Xray policy enforcing ISO/SAE 21434 TARA risk levels
# Maps CVSS Base Score to TARA risk level
{
  "name": "iso21434-security-gate",
  "rules": [
    {
      "name": "critical-cve-block",
      "criteria": {"min_severity": "Critical"},
      "actions": {
        "fail_build": true,
        "block_release_bundle_distribution": true
      }
    },
    {
      "name": "high-cve-notify",
      "criteria": {"min_severity": "High"},
      "actions": {
        "notify_email": ["cybersecurity-team@company.com"]
      }
    }
  ]
}
```

Chapter 10: Automotive CI/CD Pipeline — Deep Dive

10.1 Pipeline Architecture for ECU Software

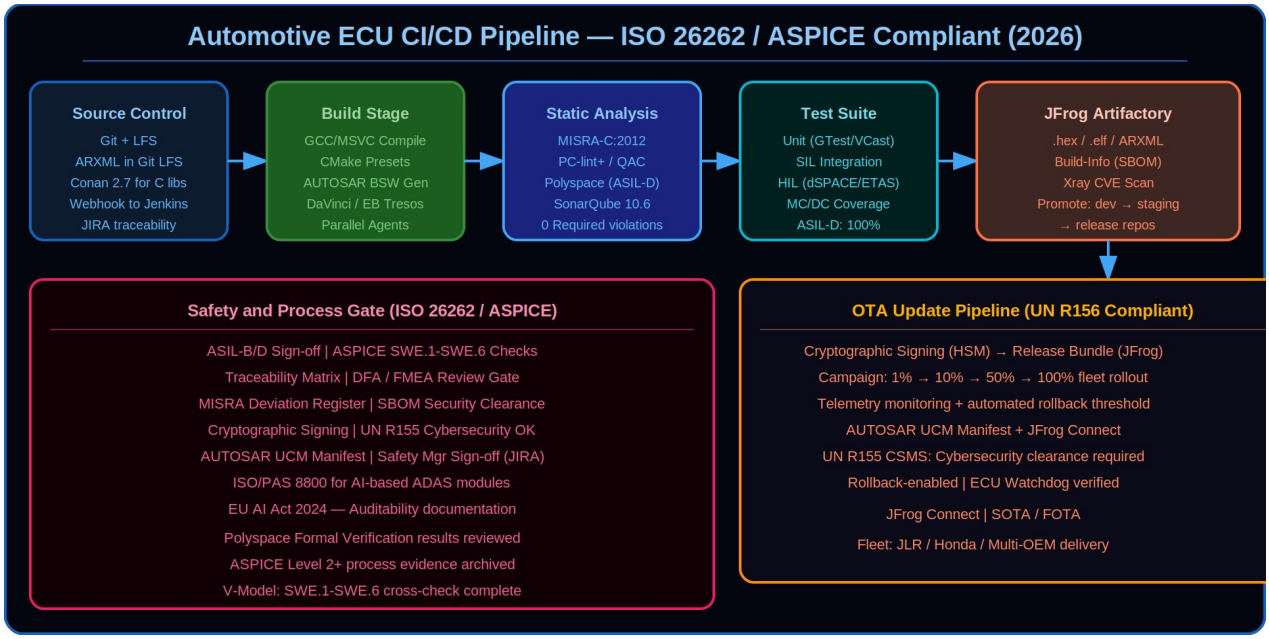

Figure 10.1 — Full Automotive Software CI/CD Pipeline: From ARXML Source to ECU Flash

The automotive ECU CI/CD pipeline is a multi-stage, safety-aware workflow that transforms C/C++ source code and AUTOSAR ARXML configurations into qualified, signed ECU software ready for OEM delivery. Unlike enterprise software pipelines, every stage must satisfy both speed and compliance objectives - ISO 26262 evidence collection, MISRA-C static analysis, and traceability to safety requirements run alongside the standard build, test, and publish stages.

The pipeline is organised into five logical phases: (1) Source validation - static analysis, MISRA-C:2012 enforcement, SonarQube quality gates; (2) Build - containerised, reproducible CMake/Conan build producing .hex, .elf, and ARXML artefacts; (3) Verification - unit tests (GTest/VectorCAST), SIL integration tests, nightly HIL regression runs; (4) Artefact management - promotion from dev-local to staging-local in JFrog Artifactory with full build-info and SBOM; (5) Release - cryptographic signing, Release Bundle creation, and OTA campaign orchestration via JFrog Connect.

> ℹ️ Pipeline-as-code (Jenkinsfile stored in the repository) ensures that every change to the build or test process is reviewed, versioned, and auditable - a direct enabler of ASPICE PA 2.1 (Work Product Management) and ISO 26262 software process evidence requirements.

10.2 Source Control for Automotive SW

10.2.1 Repository Structure

Automotive SW repositories require careful structuring to handle the diversity of artefacts: C source code, ARXML configurations, calibration data, model-based design files, and documentation.

```
automotive-ecu-repo/
├── src/
│   ├── appl/          # Application SWCs (C code)
│   ├── bsw/           # BSW stubs / callouts (C code)
│   └── integration/   # AUTOSAR integration code
```

```
├── cfg/
│
├── arxml/         # AUTOSAR configuration (Git LFS)
│
└── calibration/   # Calibration datasets (.a2l, .hex)
├── models/            # Simulink/Targetlink models (Git LFS)
├── tests/
│
├── unit/          # GTest unit test suites
│
├── integration/   # SIL integration tests
│
└── hil/           # HIL test scripts (Python/CAPL)
├── ci/
│
├── Jenkinsfile    # Main pipeline definition
│
├── conanfile.txt  # C++ dependency manifest
│
└── docker/        # Build toolchain Dockerfiles
├── docs/              # Safety documentation (versioned)
└── CMakeLists.txt     # Top-level CMake build system
```

10.3 Build Toolchain Containerization

Containerizing automotive build toolchains solves the 'works on my machine' problem endemic in automotive projects, where toolchain versions (compiler, AUTOSAR tool, testing framework) directly affect safety qualification. Docker containers provide bit-for-bit reproducible builds.

```
# Dockerfile: Automotive C/C++ build toolchain
FROM ubuntu:22.04 AS automotive-builder
LABEL maintainer="ci-team@company.com"
LABEL autosar-tools-version="4.4.0"
RUN apt-get update && apt-get install -y \
    gcc-arm-none-eabi=10.3-2021.10 \
    cmake=3.22.1 \
    ninja-build \
    gcovr lcov \
    python3 python3-pip \
  && pip install conan==2.2.0 \
  && rm -rf /var/lib/apt/lists/*

# Install PC-lint (MISRA checker)
COPY tools/pclint/ /usr/local/pclint/
ENV PATH="/usr/local/pclint:${PATH}"
# Verify toolchain versions for qualification record
RUN arm-none-eabi-gcc --version && conan --version
WORKDIR /workspace
```

10.4 MISRA-C and Static Analysis Integration

MISRA-C (Motor Industry Software Reliability Association) is the de facto C coding standard for safety-critical automotive software. MISRA C:2012 defines 143 guidelines (rules and directives). Compliance is mandatory for ISO 26262 ASIL B/C/D software. CI pipelines must enforce MISRA checks on every build.

10.4.1 PC-lint Plus / QAC Integration in Jenkins

```
stage('MISRA-C:2012 Static Analysis') {
agent { docker { image 'automotive-builder:latest' } }
steps {     sh '''
# Run PC-lint with MISRA C:2012 ruleset
     find src/ -name '*.c' | xargs pclint \\          +v \\          -DMISRA_C_2012 \\
-si4 -sl4 \\
       misra_c_2012.lnt \\         -format='%f(%l): [%n] %t %m' \\         -max_errors=0
\\
       2>&1 | tee misra-report.txt
```

```
# Exit code check
    if grep -q 'Required' misra-report.txt; then
      echo 'MISRA Required violations found!'; exit 1
    fi    '''   }
post {    always {
    archiveArtifacts 'misra-report.txt'
      publishHTML([reportDir: '.', reportFiles: 'misra-report.txt', reportName: 'MISRA
Report'])    }   } }
```

10.5 Testing Levels in Automotive CI/CD

10.5.1 Software-in-the-Loop (SIL) Testing

SIL testing executes the ECU software on a development workstation or CI server, simulating the hardware interface through software stubs. SIL enables rapid automated testing with no hardware dependency, making it ideal for CI pipelines. Frameworks: VectorCAST, LDRA, Cantata, and open-source options like Google Test with hardware abstraction layers.

10.5.2 Hardware-in-the-Loop (HIL) Testing

HIL testing connects actual ECU hardware to a real-time simulation environment (e.g., dSPACE SCALEXIO, NI HIL, ETAS LABCAR) that simulates the vehicle environment. HIL test automation integrates into the CI/CD pipeline as a longer-running stage triggered on develop branch merges or nightly builds, providing confidence before release.

```
# Python HIL test automation (simplified)
import hil_framework as hil

def test_abs_activation():
    """Verify ABS activates within 50ms of wheel slip detection."""
    hil.set_wheel_speed('FL', 120)  # km/h
    hil.set_wheel_speed('FR', 0)    # Simulate locked wheel
    hil.start_simulation()
    result = hil.wait_for_signal('ABS_ACTIVE', timeout_ms=100)
     assert result.activated_at_ms < 50, f"ABS latency {result.activated_at_ms}ms > 50ms
SLA"
    hil.verify_no_dtc_set(['C0100', 'C0101'])  # No spurious DTCs
```

10.6 JFrog Artifactory for Automotive Artifact Management

10.6.1 Repository Layout for Automotive SW

```
# JFrog Artifactory repository structure automotive-dev-local/          # In-development
builds
├── my-ecu/1.0.0-SNAPSHOT.42/
│
├── my-ecu-1.0.0-42.hex
│
├── my-ecu-1.0.0-42.elf
│
├── arxml-config.tar.gz
│
└── build-info.json       # SBOM, test results, MISRA status automotive-staging-local/
#  Promoted builds (passed all CI tests) automotive-release-local/       # OEM-release
builds (passed HIL + safety gate)
├── my-ecu/v1.0.0/
│
├── my-ecu-v1.0.0-signed.hex    # Cryptographically signed
│
├── sbom-my-ecu-v1.0.0.spdx    # SPDX SBOM for UN R155
│
└── safety-evidence-pack.zip   # ISO 26262 evidence
```

10.7 OTA Update Pipeline

Over-the-Air (OTA) software updates for vehicles require a secure, reliable, and rollback-capable delivery mechanism. The complete OTA pipeline integrates JFrog, AUTOSAR UCM, and UN R156 compliance.

10.7.1 OTA Pipeline Stages

7. Software Package Build: ECU .hex + AUTOSAR UCM manifest generated by CI pipeline
8. Security Scan: JFrog Xray scans the update package for CVEs and license issues
9. Cryptographic Signing: Update package signed with HSM-stored private key
10. Release Bundle Creation: JFrog Distribution creates signed, immutable Release Bundle
11. Campaign Creation: Campaign created in OTA management system (JFrog Connect / custom)
12. Staged Rollout: 1% → 10% → 50% → 100% fleet rollout with telemetry monitoring
13. Rollback Trigger: Automated rollback if error rate>threshold or manual abort

> ⚠️ UN R156 requires documentation of the update mechanism, rollback capability, protection against incomplete updates, and prevention of unauthorized software installation. These requirements must be reflected in the CI/CD pipeline's signing, distribution, and monitoring stages.

10.8 ISO 26262 CI/CD Safety Gate Checklist

Before any artifact can be promoted from staging to release, the following automated and human gates must pass:

- ✓ **All unit tests pass with 100% MC/DC coverage (ASIL-D) or ≥85% line coverage (ASIL-B)**
- ✓ MISRA-C:2012Required violations = 0; Advisory violations documented in deviation register
- ✓ No new MISRA-C deviations compared to baseline without approved deviation justification
- ✓ SIL integration test suite passes (100%)
- ✓ HIL regression test suite passes (100%)
- ✓ Polyspace / Astrée formal verification results within acceptable bounds
- ✓ JFrog Xray: 0 Critical/High unaccepted CVEs in dependencies
- ✓ SBOM generated and reviewed; all open-source licenses approved
- ✓ Requirement traceability matrix complete: every safety requirement has ≥1 test
- ✓ DFA (Dependent Failure Analysis) reviewed for ASIL decomposition points
- ✓ Safety Manager sign-off documented in JIRA/DOORS
- ✓ Artifact cryptographically signed and stored in JFrog release repository

Chapter 11: Real-World Case Studies

11.1 Case Study 1: EV Powertrain CI/CD Migration

A Tier-1 supplier managing 12 ECUs across an EV powertrain system (BMS, VCU, OBC, DCDC converters) migrated from a manual 14-day release cycle to a CI/CD pipeline. The team spent 6 months implementing Jenkins pipelines, Git LFS for ARXML, and JFrog Artifactory as the central artifact repository.

Results Achieved

- **Release cycle reduced from 14 days to 4 hours for individual ECU software packages**
- MISRA violation escape rate dropped from 12 per release to 0 (gates enforced in CI)
- Build reproducibility: from ~60% (dependency drift) to 100% (Conan + JFrog)
- OEM audit preparation time reduced from 2 weeks to 3 days (JFrog build info + SBOM)
- HIL regression: automated nightly runs replaced 2 manual test engineers for regression

11.2 Case Study 2: ADAS ML Model CI/CD

An OEM's ADAS division building a lane-keeping assist system (LKAS) implemented MLOps using a Jenkins-based CI/CD pipeline for model training and deployment. Key challenge: model retraining triggered by data drift required the same rigour as software updates — ISO/PAS 8800 compliance with complete audit trail.

Pipeline Architecture

- **Data Version Control: DVC with JFrog Artifactory as DVC remote (S3-compatible)**
- Training: Distributed PyTorch on Kubernetes; MLflow experiment tracking
- Model Validation: 10,000-scenario CARLA simulation regression suite in CI
- Model Registry: JFrog Artifactory generic repository with model metadata properties
- Deployment: ONNX model quantization → TensorRT optimization →ECU hex file build
- Monitoring: Evidently AI drift detection + Grafana dashboard; auto-retrain trigger

11.3 Case Study 3: Multi-OEM Delivery with JFrog Distribution

A global Tier-1 supplier delivering the same AUTOSAR-based body controller software to 3 different OEMs (with different configurations, calibrations, and software variants) implemented JFrog Distribution to manage OEM-specific Release Bundles. Each OEM receives a signed, immutable bundle with only their approved software variant, preventing cross-contamination of configurations. The challenge: a single source codebase, compiled with 3 different AUTOSAR DaVinci configurations, required separate security sign-offs from each OEM before distribution.

Pipeline Architecture

- Single Git monorepo with variant-specific ARXML configs under cfg/oem-a/, cfg/oem-b/, cfg/oem-c/
- Matrix Jenkins pipeline builds all 3 OEM variants in parallel using parameterised Dockerised toolchains
- Shared test suites run per variant; OEM-specific calibration checks run against each OEM's A2L files

- JFrog Artifactory stores variant builds under automotive-staging-local/bcc/{variant}/{build}/
- JFrog Xray policy per OEM enforces variant-specific approved open-source license lists
- JFrog Distribution Release Bundle per OEM: signed with OEM-specific key, contains only that OEM's variant
- Release Bundle distributed to OEM's private Artifactory Edge node - no cross-OEM data exposure

Results Achieved

- OEM configuration cross-contamination incidents: reduced from 3 per year to 0 (release bundles enforce isolation)
- Release preparation time: from 5 days (manual packaging per OEM) to 4 hours (automated matrix pipeline)
- Audit evidence generation: OEM-specific SBOM and traceability reports auto-generated per bundle
- Supply chain compliance: all 3 OEMs accepted JFrog build-info as ASPICE SWE.6 evidence artefacts
- ISO/SAE 21434 TARA coverage: automated Xray scan results mapped to cybersecurity risk register per OEM variant

> ℹ️ JFrog Distribution Release Bundles map directly to UN R156 RXSWIN (Software Identification Number) requirements: each bundle carries an immutable identifier linking the OEM software variant to its type-approved configuration, satisfying SUMS traceability requirements end-to-end.

Appendix A: Tool Version Reference (June 2026)

Tool	Version (June 2026)	Notes
Jenkins LTS	2.504.x	Java 21 runtime required; Kubernetes-native agents; JCasC 1.56.x
Git	2.47.x	SHA-256 object naming; Git LFS 3.6.x; signed commits enforcement
JFrog Artifactory	7.90.x	PostgreSQL 16 backend; Helm chart 107.x; Release Bundle v2 support
JFrog Xray	3.108.x	Advanced Security: contextual analysis, AI fix suggestions, secrets detection
Docker CE	27.x	containerd 2.x; Buildkit 0.16.x; Docker Scout for runtime monitoring
Kubernetes	1.33.x	Gateway API GA; sidecar containers API stable; CRI-O as recommended runtime
Conan	2.7.x	conan2 API only; full JFrog Artifactory Conan v2 repo support
MLflow	2.16.x	AI Gateway; LLM Evaluate; Unity Catalog integration; Agent monitoring
CMake	3.31.x	CMake Presets 9; --fresh flag; package-lock.cmake for reproducibility
Prometheus	2.55.x	OTLP native receiver GA; remote write 2.0; agent mode for reduced memory
SonarQube CE	10.8.x	Java 21; AI-powered code fix suggestions; 30+ supported languages
Terraform	1.9.x / OpenTofu 1.8.x	OpenTofu is a CNCF-incubating fork (graduated incubation 2024); full compatibility with HCL configs
Argo CD	2.13.x	ApplicationSet controller; GitOps standard; multi-cluster management
Helm	3.17.x	OCI registry support stable; Helm Drift Detection; Sigstore chart signing
OpenTelemetry Collector	0.115.x	OTLP/gRPC + OTLP/HTTP; Prometheus exporter; Kafka exporter for automotive

Appendix B: Glossary (Key Terms and Acronyms)

Term	Definition
ARXML	AUTOSAR XML — the configuration and metadata file format for AUTOSAR BSW configuration
ASIL	Automotive Safety Integrity Level — A through D (D being most stringent) per ISO 26262:2018
ASPICE	Automotive SPICE — Software Process Improvement and Capability dEtermination framework (v4.0, 2023)
BSW	Basic Software — AUTOSAR standardised middleware layer below the application layer
Canary Deploy	Deployment pattern releasing to a small percentage of users/vehicles before full fleet rollout
CSMS	Cybersecurity Management System — required by UN R155 for vehicle type approval
CycloneDX	OWASP SBOM standard (v1.6 in 2026) for software component inventory in JSON/XML
DORA	DevOps Research and Assessment — Google programme studying DevOps metrics (State of DevOps 2025)
DVC	Data Version Control — Git-compatible versioning for ML datasets and model artefacts
EU AI Act	Regulation (EU) 2024/1689 — AI regulation classifying ADAS as high-risk AI systems
GitOps	Practice of using Git as single source of truth for infrastructure and deployment config
HARA	Hazard Analysis and Risk Assessment — ISO 26262 safety analysis methodology
HIL	Hardware-in-the-Loop — testing real ECU hardware against a real-time simulated environment
IaC	Infrastructure as Code — managing infrastructure through machine-readable version-controlled files
IDP	Internal Developer Platform — self-service developer toolchain managed by Platform Engineering teams
ISO/PAS 8800	ISO/PAS 8800:2024 — Safety of AI-based systems in road vehicles (released February 2024)
MCAL	Microcontroller Abstraction Layer — AUTOSAR hardware abstraction for peripheral drivers
MISRA	Motor Industry Software Reliability Association — C:2012 coding standard for safety SW
MLOps	Machine Learning Operations — DevOps principles applied to the ML model lifecycle
LLMOps	Large Language Model Operations — operational management of LLMs and RAG pipelines
OTA	Over-the-Air — wireless software/firmware update for embedded systems
SBOM	Software Bill of Materials — inventory of all SW components and dependencies
SIL	Software-in-the-Loop — testing ECU SW on host PC with simulated hardware

	interfaces
SOME/IP	Scalable service-Oriented MiddlewarE over IP — AUTOSAR service communication protocol
SOTIF	Safety of the Intended Functionality — ISO 21448, covers residual risk in ADAS systems
SPDX	Software Package Data Exchange — Linux Foundation SBOM standard (v2.3 and v3.0, April 2024)
SWC	Software Component — AUTOSAR application unit with defined ports and interfaces
TBD	Trunk-Based Development — all developers commit to main/trunk with short-lived branches
UCM	Update and Configuration Management — AUTOSAR module for OTA software update handling

Appendix C: Standards and References (2026)

Standards and Regulatory Documents

- **ISO 26262:2018 — Road Vehicles: Functional Safety (Parts 1-12)**
- ISO/SAE 21434:2021 — Road Vehicles: Cybersecurity Engineering
- ISO/PAS 8800:2024 — Safety of AI-based systems in road vehicles
- ISO 21448:2022 (SOTIF) — Safety of the Intended Functionality
- AUTOSAR Classic Platform R24-11 Specification — www.autosar.org (November 2024)
- AUTOSAR Adaptive Platform R24-11 Specification — www.autosar.org (November 2024)
- ASPICE PAM 4.0 (2023) — VDA Quality Management Center
- UNECE WP.29 Regulation 155 (Cybersecurity) and Regulation 156 (Software Updates)
- EU AI Act (2024) — Regulation (EU) 2024/1689, fully enforced August 2026
- NIST AI RMF 1.0 (2023) — AI Risk Management Framework

Books

- **'The DevOps Handbook' (2nd Ed) — Gene Kim, Jez Humble, Patrick Debois, John Willis (IT Revolution, 2021)**
- 'Continuous Delivery' — Jez Humble&David Farley (Addison-Wesley, 2010)
- 'Accelerate' — Forsgren, Humble, Kim (IT Revolution, 2018) — DORA foundational research
- 'Designing Machine Learning Systems' — Chip Huyen (O'Reilly, 2022)
- 'Platform Engineering' — Luca Galante, Abby Bangser (O'Reilly, 2024)
- 'Site Reliability Engineering' — Betsy Beyer et al. (Google/O'Reilly, 2016)
- 'The Phoenix Project' (3rd Ed) — Gene Kim, Kevin Behr, George Spafford (IT Revolution, 2018)
- 'AI Engineering' — Chip Huyen (O'Reilly, 2025)

Online Resources

- **DORA State of DevOps 2025 Report — dora.dev**
- JFrog Documentation 7.90 — jfrog.com/help
- Jenkins LTS 2.504 Documentation — www.jenkins.io/doc
- Git 2.47 Documentation — git-scm.com/book
- MLflow 2.16 Documentation — mlflow.org/docs/latest
- CNCF Cloud Native Landscape 2026 — landscape.cncf.io
- AUTOSAR — www.autosar.org
- MISRA C:2012 — www.misra.org.uk
- EU AI Act — eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX:32024R1689

---:--

End of Handbook

Compiled by Ashish Kumar | Senior Technical Leader&Associate Engineering Manager

KPIT Technologies Limited, Bengaluru | M.Tech IIT(ISM) Dhanbad | PGC-GM IIM Nagpur

“Ship fearlessly. Build correctly. Drive safely.”

Edition 2026 | June 2026 | Bengaluru, India